

Recurrent Neural Network

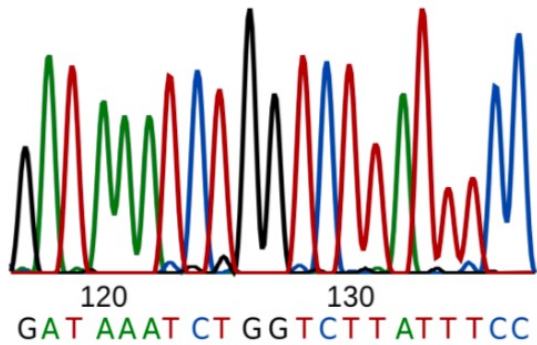
01

Recurrent Neural Network

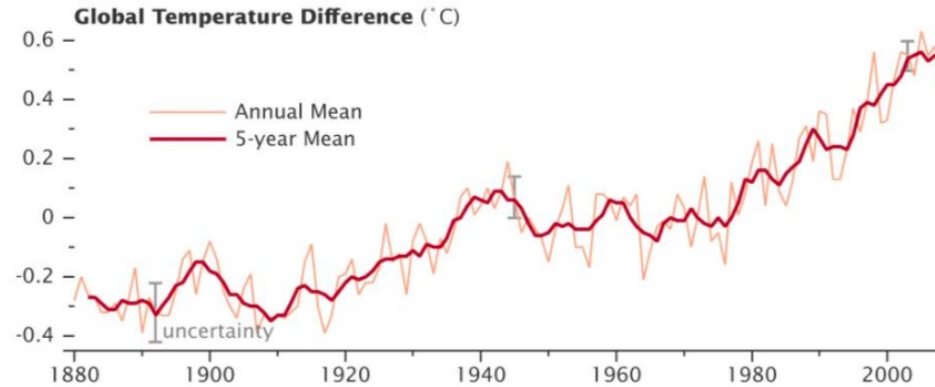
RNN (Recurrent Neural Network)

- RNN(Recurrent Neural Network)은 CNN과 함께 대표적인 딥러닝 모델
- 시계열 데이터와 같은 순차 데이터(Sequential Data) 처리를 위한 모델

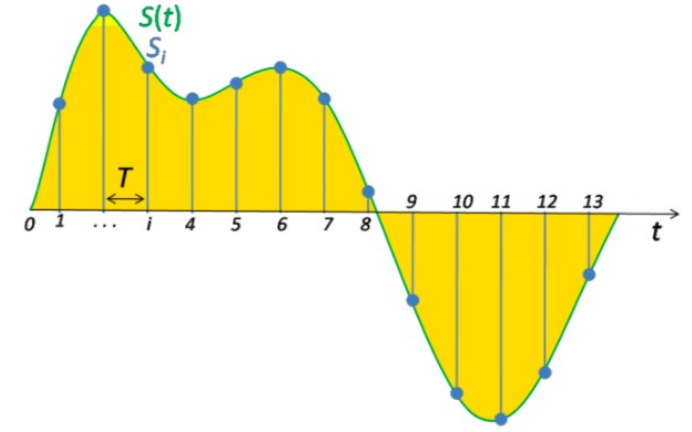
Sequential Data (순차 데이터)



DNA 염기 서열
(Sequential Data)



세계 기온 변화
(Temporal Sequence)



샘플링된 소리 신호
(Time Series)

- 순서(Order)를 가지고 있는 데이터
- 시계열 데이터, 자연어 데이터 등이 대표적

Sequential Data

- 순서가 있는 연속적인 데이터
- 각 시점(Time step)의 데이터가 이전 시점의 데이터와 독립적이지 않다

예를 들어, 일일 주가 데이터 세트가 있다면

1월 3일의 주가는 1월 2일의 주가의 영향을 받았을 가능성이 높다.

1월 2일의 주가는 1월 1일의 주가의 영향을 받았을 가능성이 높다.

즉 특정 시점 t 에서의 데이터가 이전 시점($t_0, t_1, \dots, t_{n-1}$)의 영향을 받는 데이터를 순차데이터라고 한다.

자연어(Natural Language)

- 우리가 대화, 서류, 채팅 등을 통해 일상적으로 사용하는 언어
- 1) 일반적으로 길이가 가변적이며 2) 순서에 따라 의미가 있음

저기 죽어 있는 **로미오**는 **줄리에트**양의 **남편**, 역시 저기 죽어 있는 **줄리에트**는 **로미오**의 성실한 **아내**였습니다.

~ (중략) ~

따님의 벅찬 슬픔을 달래고자

~ (중략) ~

납골당에서 **그녀**를 구해 내려고 왔지요.

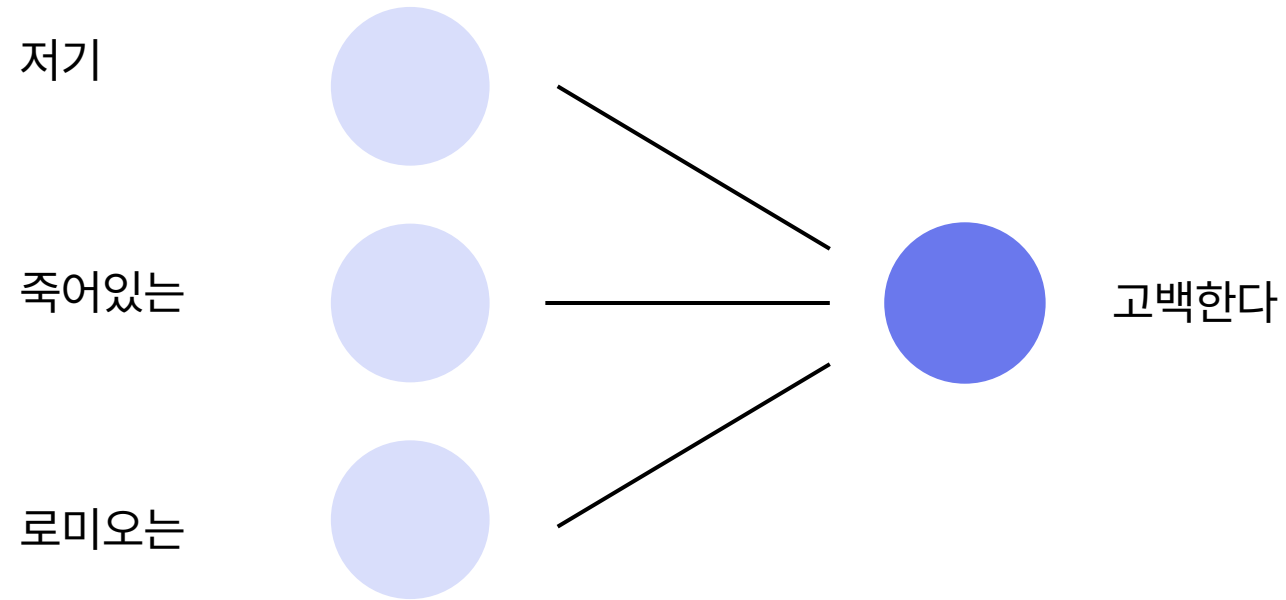
~ (중략) ~

또한 이게 **그**의 천명 탓이니까

~ (중략) ~

그녀는 실망한 나머지 따라나오려 하지 않고

RNN이 필요한 이유



자연어 문장을 **기존 MLP 모델에 적용시키기에는 한계가 있음**

저기 죽어 있는 로미오는 줄리에트양의 남편, 역시 저기 죽어 있는 줄리에트는 로미오의 성실한 아내였습니다.

~ (중략) ~

따님의 벅찬 슬픔을 달래고자

~ (중략) ~

납골당에서 그녀를 구해 내려고 왔지요.

~ (중략) ~

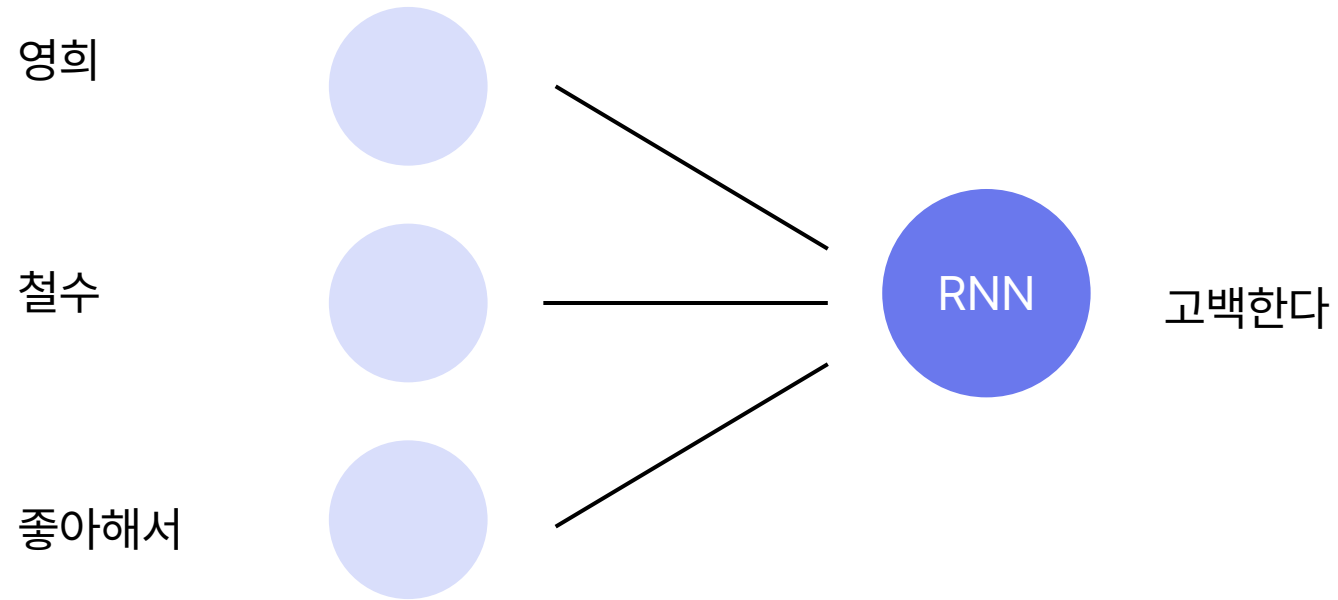
또한 이게 그의 천명 타이니까

~ (중략) ~

그녀는 실망한 나머지 따라나오려 하지 않고

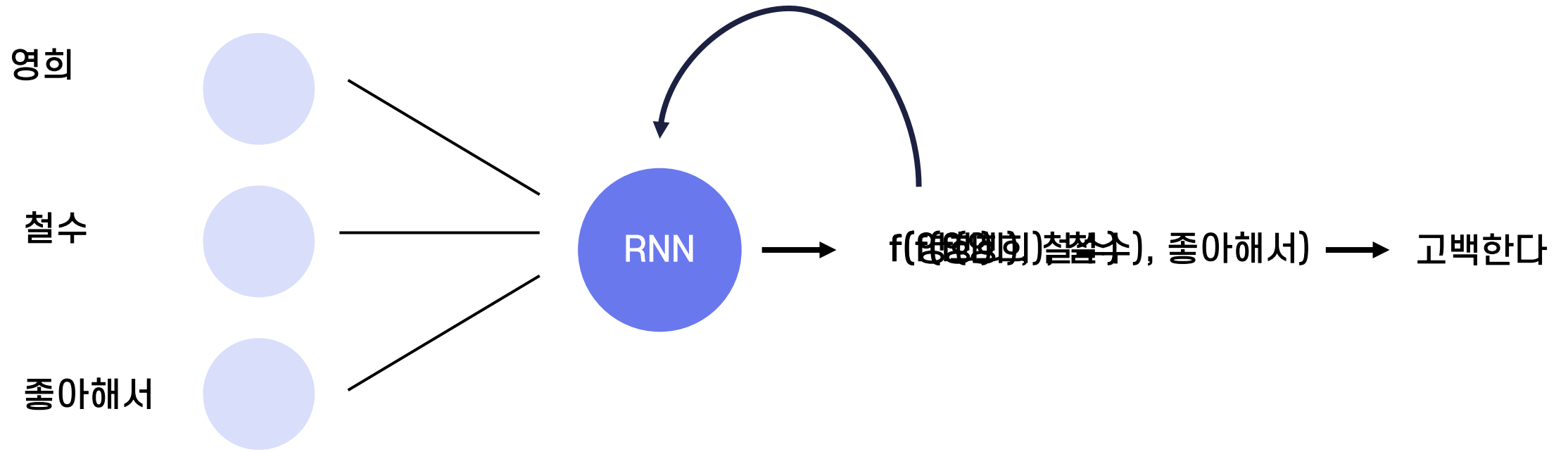
자연어 문장을 기존 CNN 모델에 적용시키기에는 한계가 있음

RNN이 필요한 이유

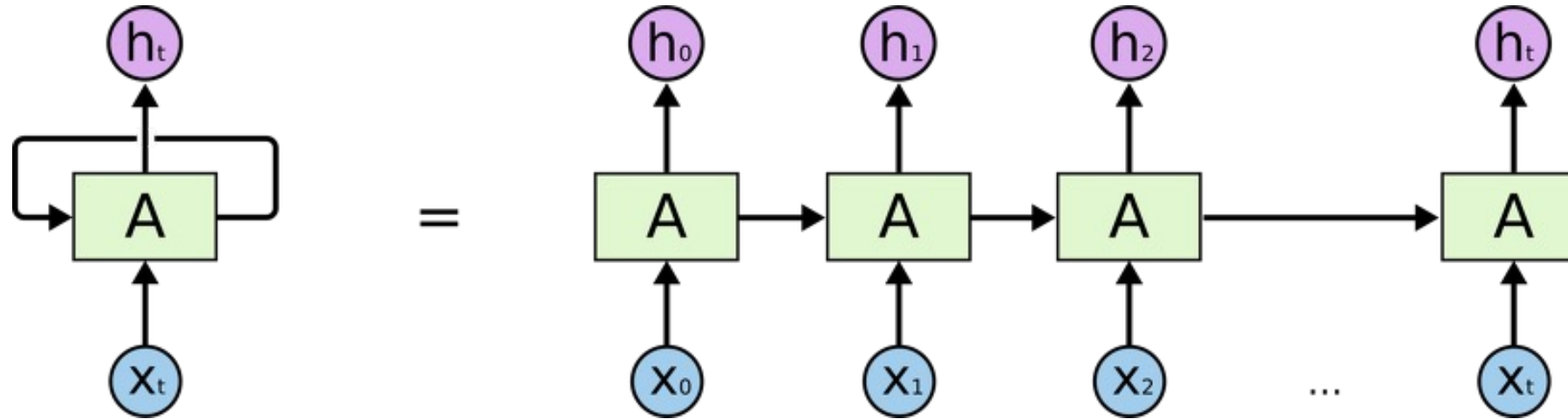


단어(입력 데이터)를 **순차적으로** 모델에 넣는다!

RNN이 필요한 이유

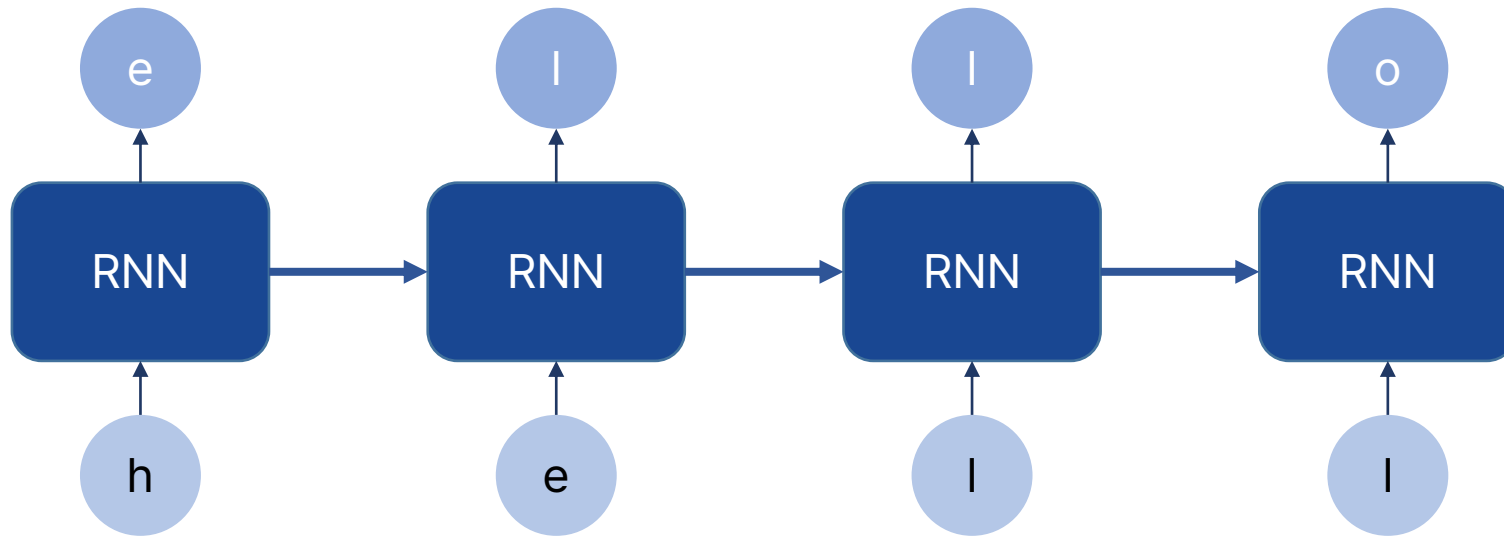


RNN의 입력값 : (1) 입력데이터 (2) RNN을 거쳐 나온 출력값 상태



- 순차 데이터 처리를 위한 딥러닝 모델이 등장
- 각 시점의 데이터를 입력으로 받아 은닉상태와 출력값((Output state, Hidden state)을 계산하는 노드를 순환 신경망의 셀(Cell)이라 한다
- 순환 신경망의 셀은 이전 시점의 은닉상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉상태 h_t 를 계산한다

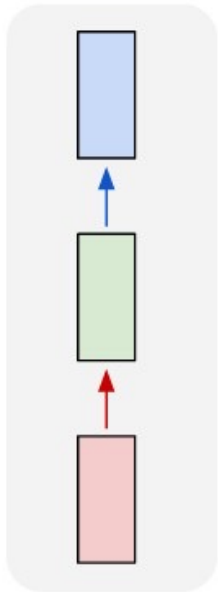
❖ RNN Character prediction 예시



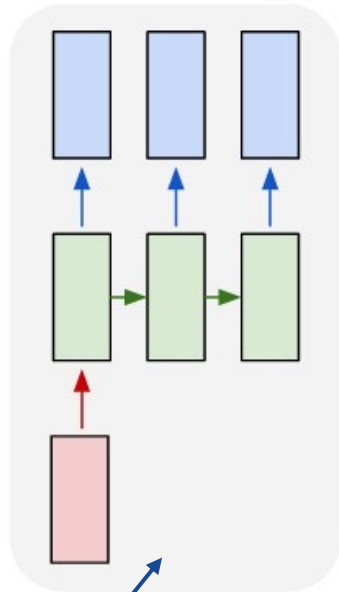
- Input : hell
- Output : o를 반환해 결과적으로 hello 출력

RNN의 종류

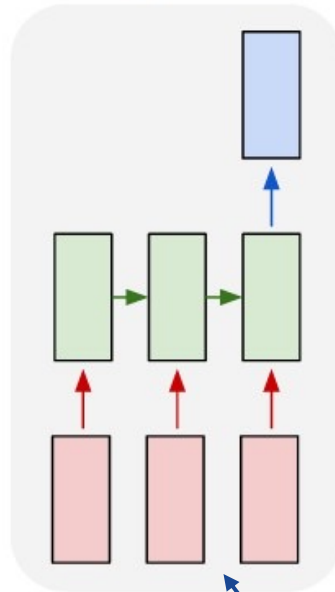
one to one



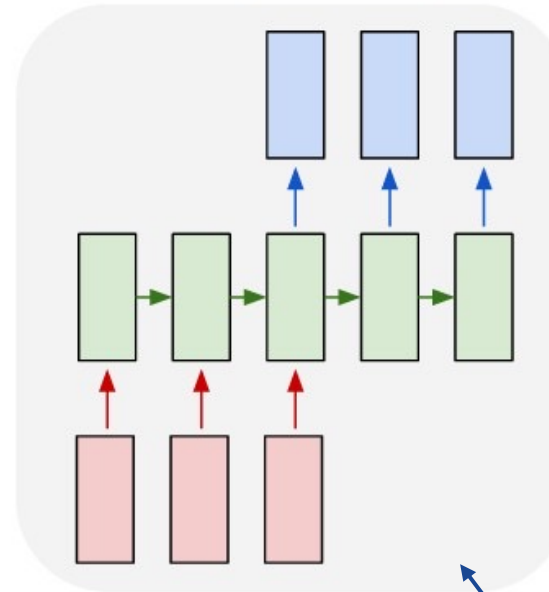
one to many



many to one



many to many



many to many

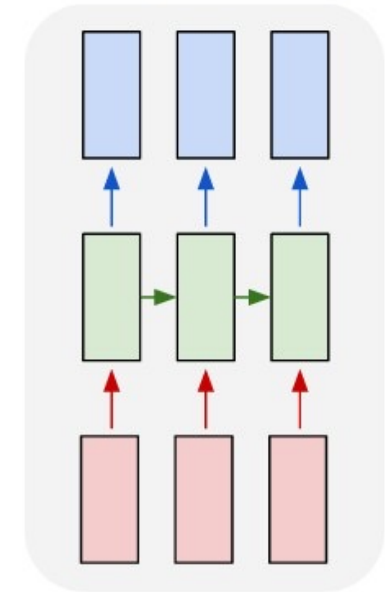
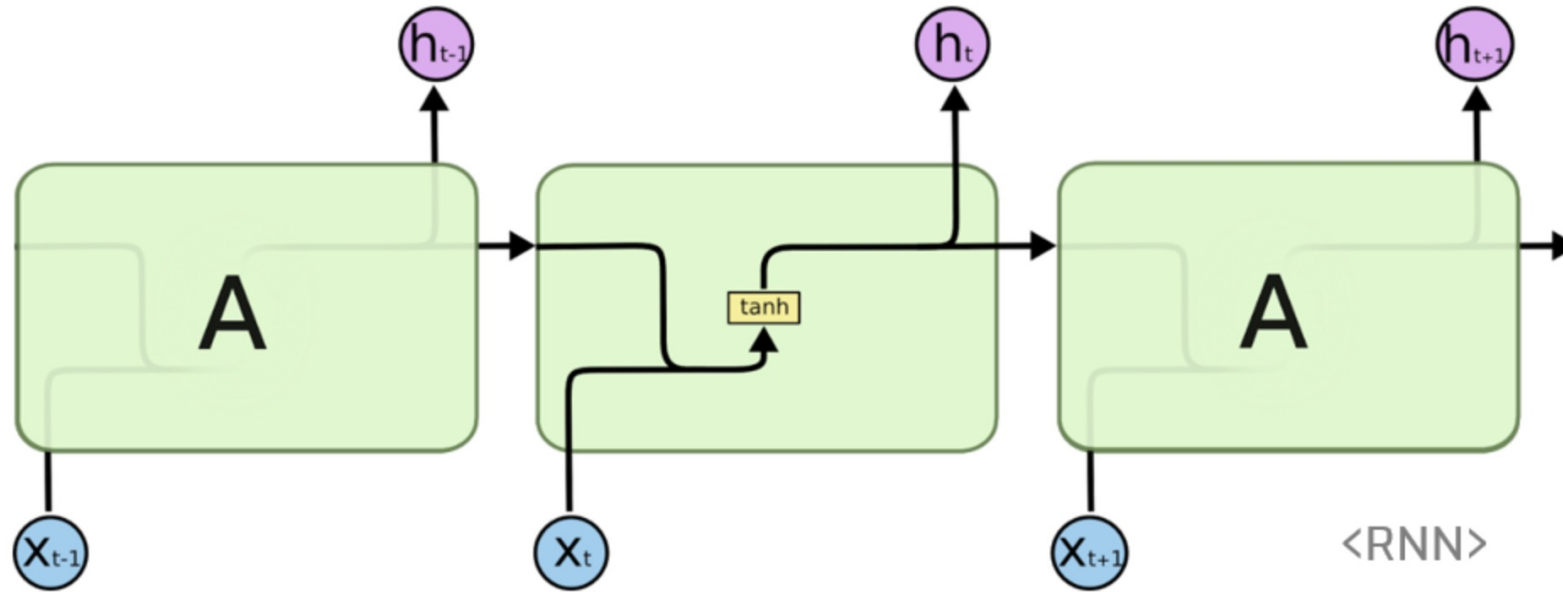


Image Captioning
(image -> sequence of words)

Sentimental Classification
(Sequence of words -> sentiment)

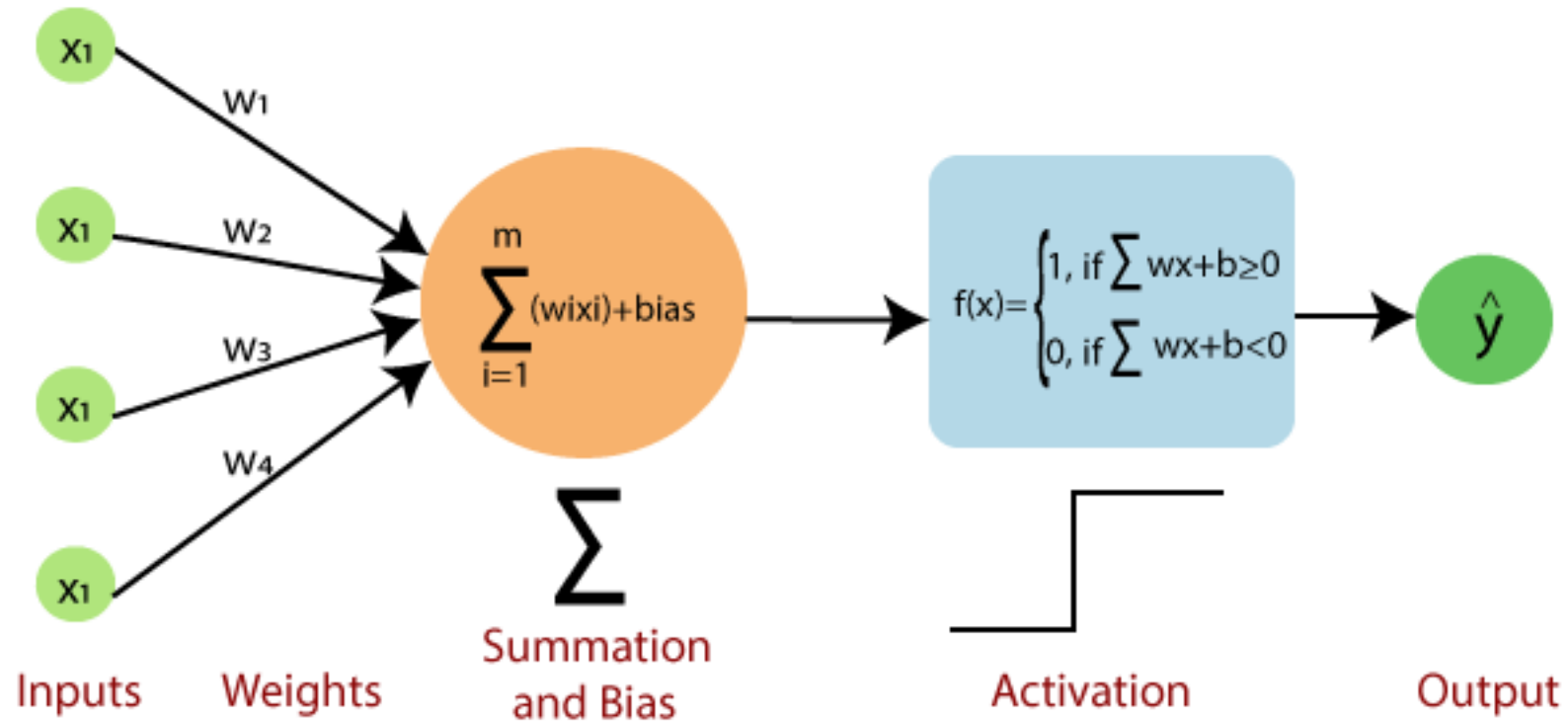
Machine Translation
(seq of words -> seq of words)

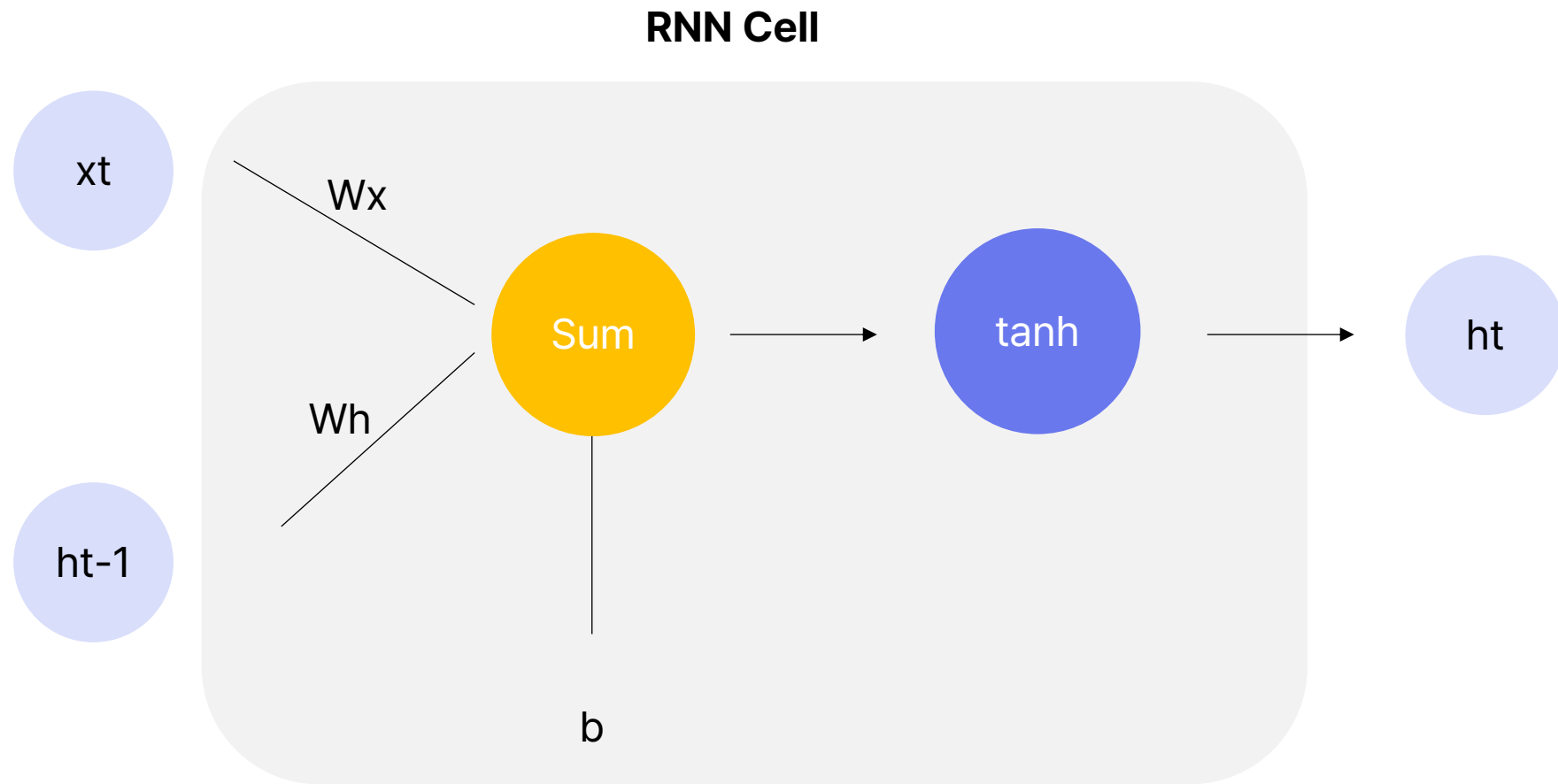
Vanilla RNN



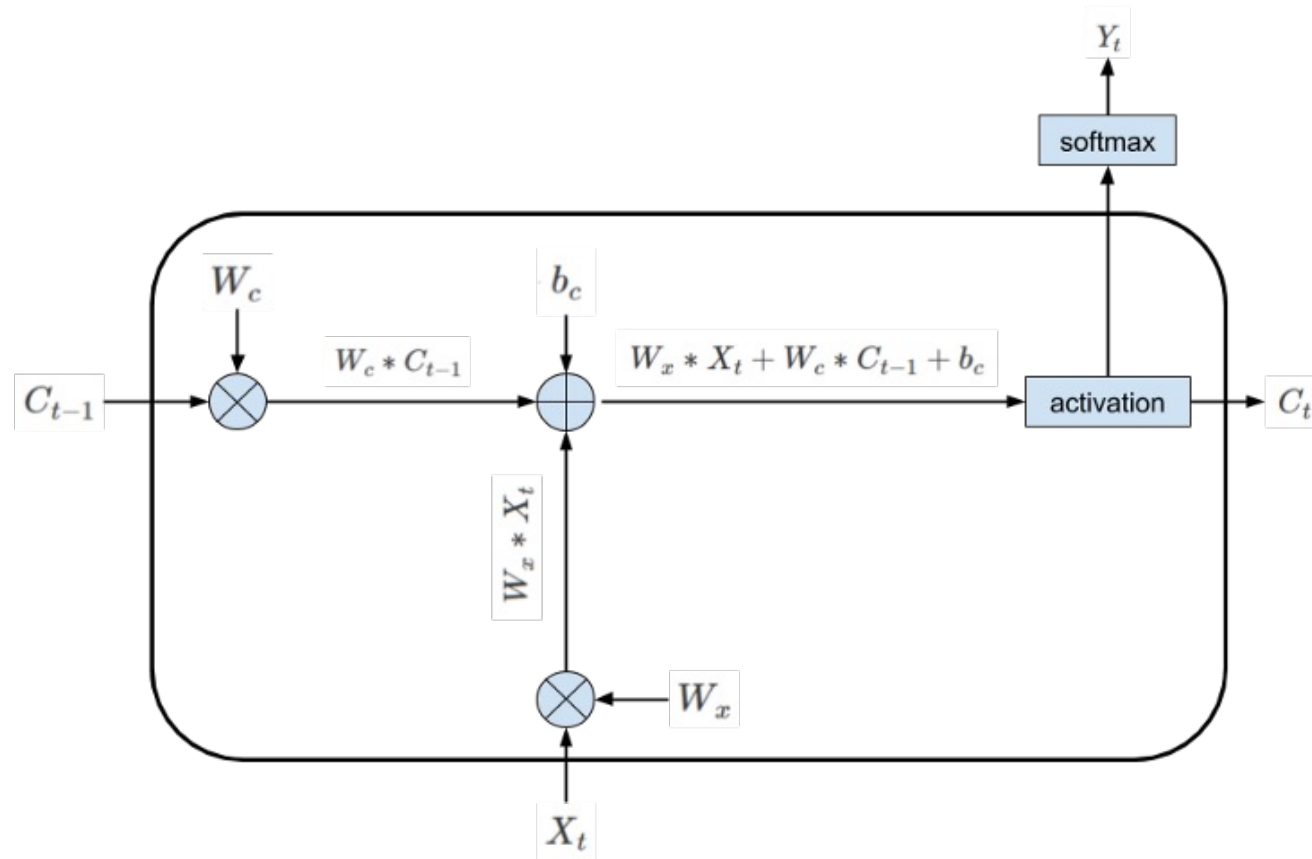
- 순환신경망은 동일한 구조의 네트워크(A)가 병렬적으로 연결되어있는 형태
- 하나의 네트워크를 보면, 이전의 네트워크에서 계산된 값(h)과 현재 input값(X)을 함께 “가중합” 해서 $\tanh$ 로 은닉층 노드(h)를 계산한다.
- 여기서 가장 중요한 핵심은 “동일한 구조”라는 의미인데, 여기서 동일한 구조라는 건, 네트워크 안의 weight의 구성까지 모두 동일하다는 것을 의미

Perceptron





Vanilla RNN



Note: W_c b_c W_x are internally stored in the RNN cell

- Input_dim = 4
- # of units (output_dim) = 5
- Seq_length = 3

$$ht = \tanh(Wx * Xt + Wh * ht-1 + bh)$$

$$Xt = [1, 0, 0, 0]$$

$$Wx = ?$$

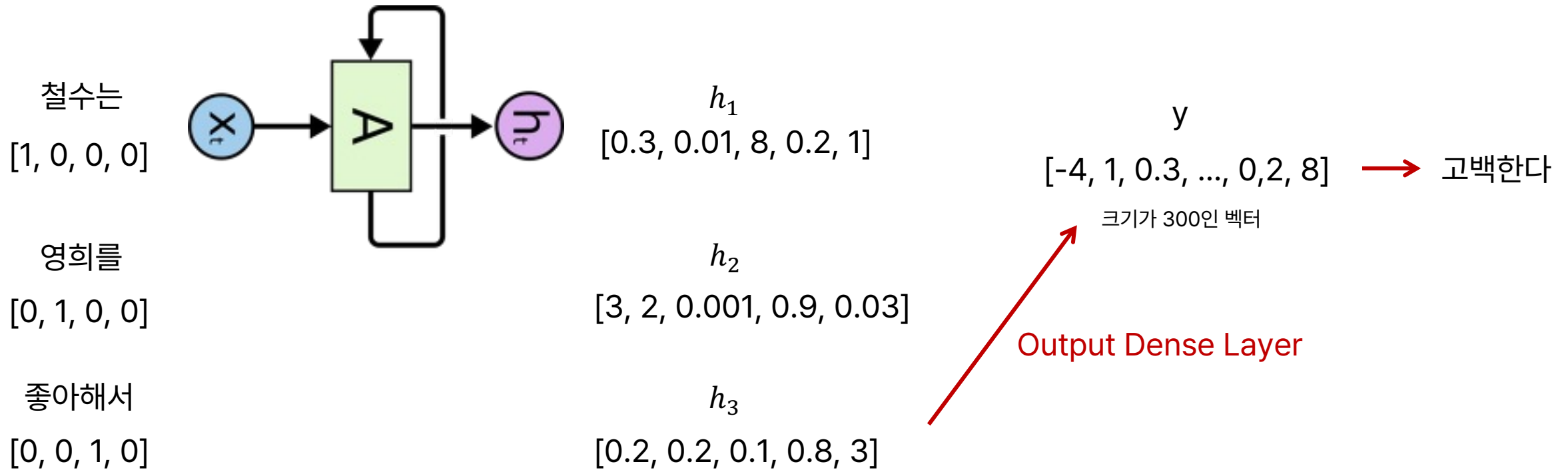
$$Wh = ?$$

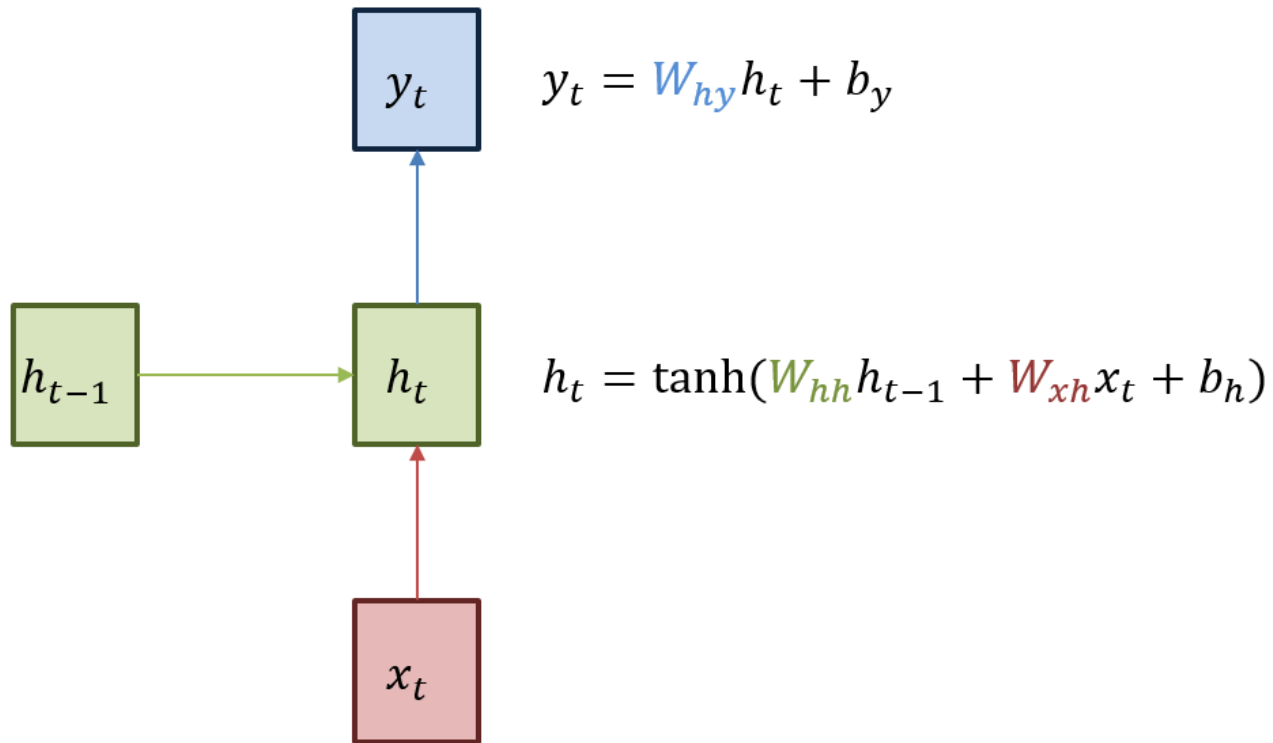
$$ht-1 = [0.1, 0.2, 0.3, 0.2, 0.1]$$

$$bh = ?$$

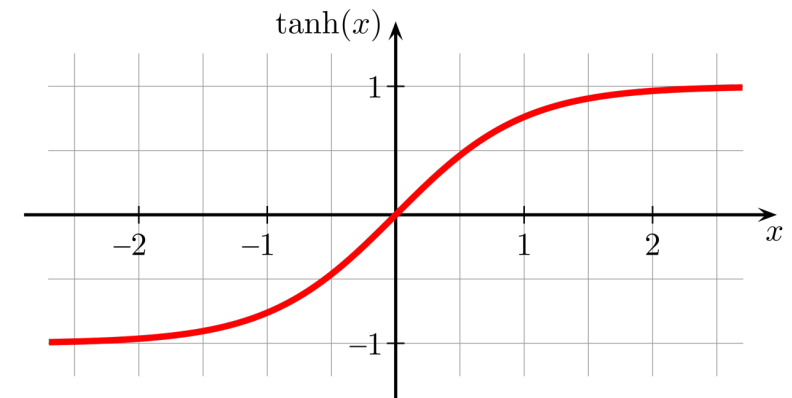
Vanilla RNN

- Input dim : 4
- Hidden dim : 5
- Output dim : 300





- 가장 간단한 형태의 RNN 모델
- 내부에 세개의 FC Layer로 구성
 - W_{hh} : hidden state(h_{t-1})를 변환하는 Layer의 가중치 행렬
 - W_{xh} : 한 시점의 입력값 (x_t) 을 변환하는 Layer의 가중치 행렬
 - W_{hy} : 한 시점의 출력값



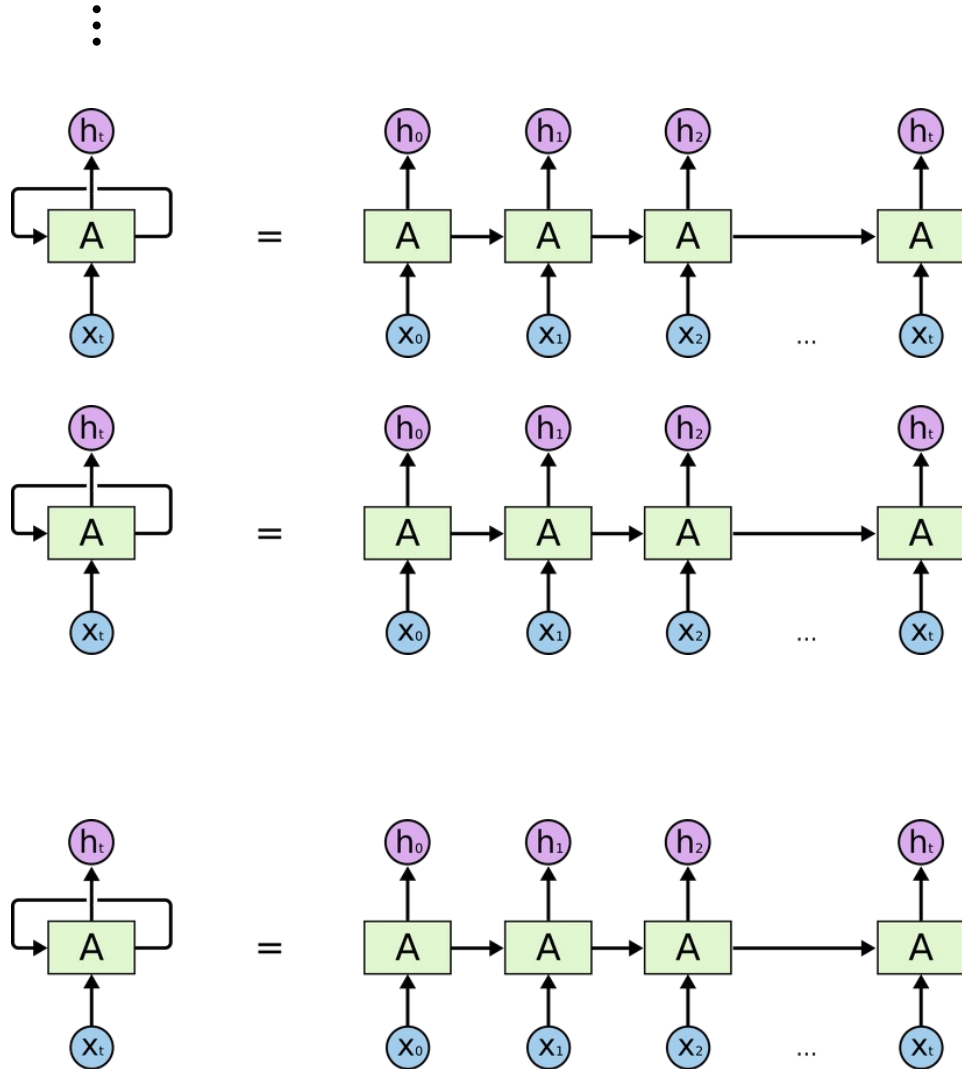
❖ Hidden state의 의미

- 특정 시점 t 까지 들어온 입력값의 상관관계나 경향성 정보를 압축해서 저장
- 모델이 내부적으로 계속 가지는 값이므로 일종의 메모리(Memory)로 볼 수 있음

❖ Parameter Sharing

- Hidden state와 출력값 계산을 위한 FC Layer에 모든 시점의 입력값이 사용
- 각각의 가중치 W_x , W_h , W_y 의 값은 하나의 층에서는 모든 시점에서 값을 동일하게 공유
- 하지만 은닉층이 2개 이상일 경우에는 각 은닉층에서의 가중치는 서로 다름

Vanilla RNN



- 옆으로 병렬적으로 길어지는 길이 :

input data의 sequence(길이)만큼

- 위로 쌓이는 높이 :

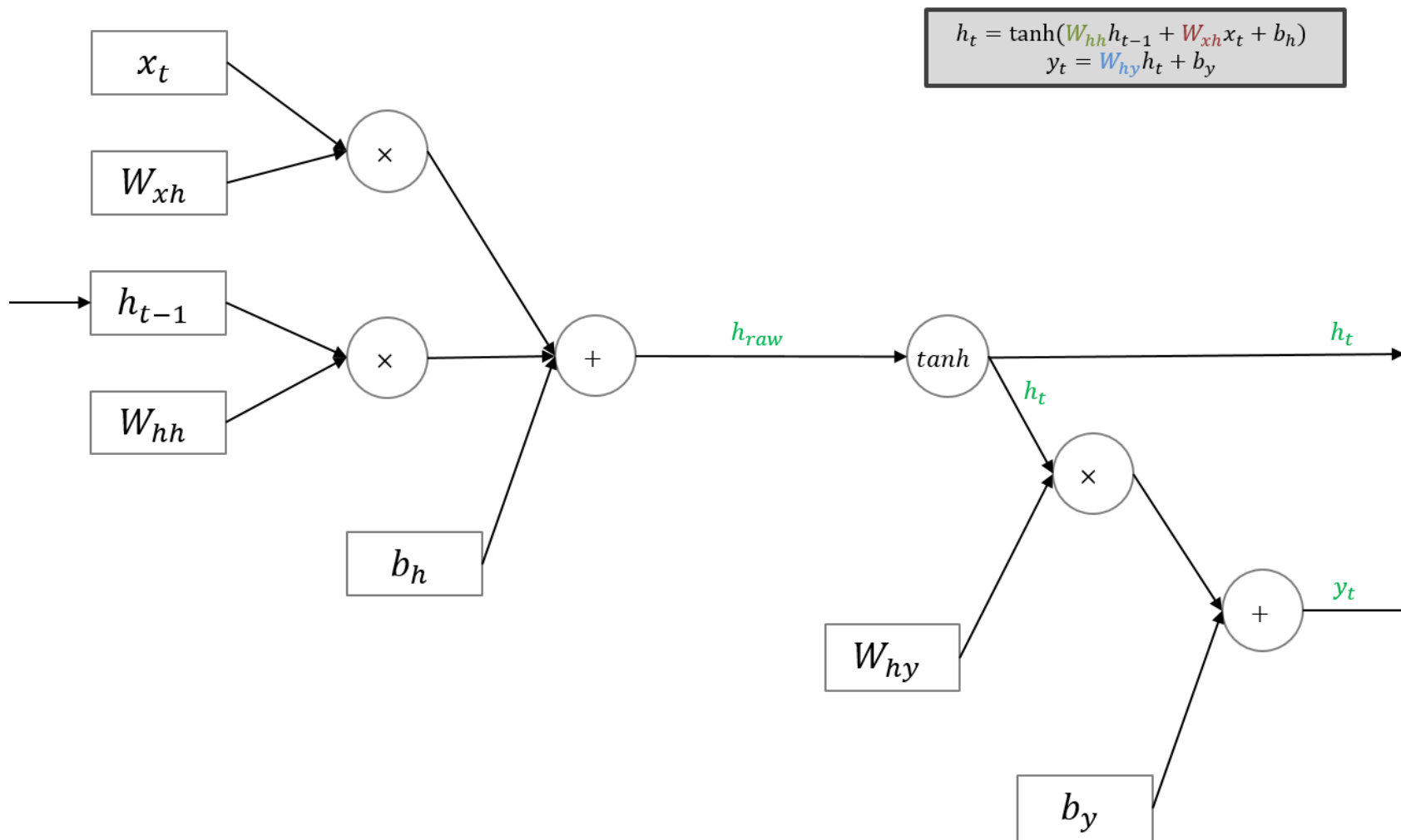
네트워크의 층 수만큼(layer의 수)

- 즉, 동일한 구조라는 건 같은 층끼리만 해당하는 것!

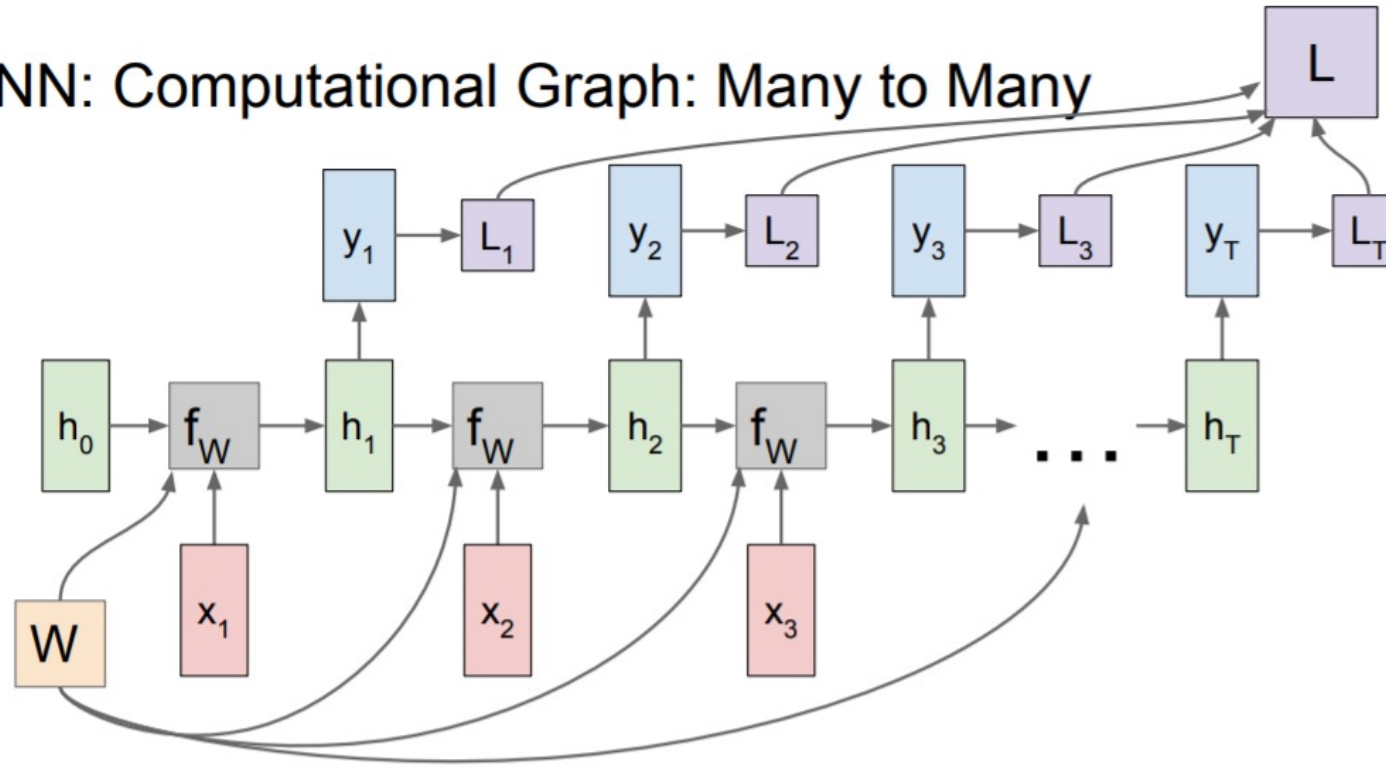
--> 같은 층에서 순환

RNN 학습 : Back-propagation Through Time (BPTT)

❖ RNN의 순전파



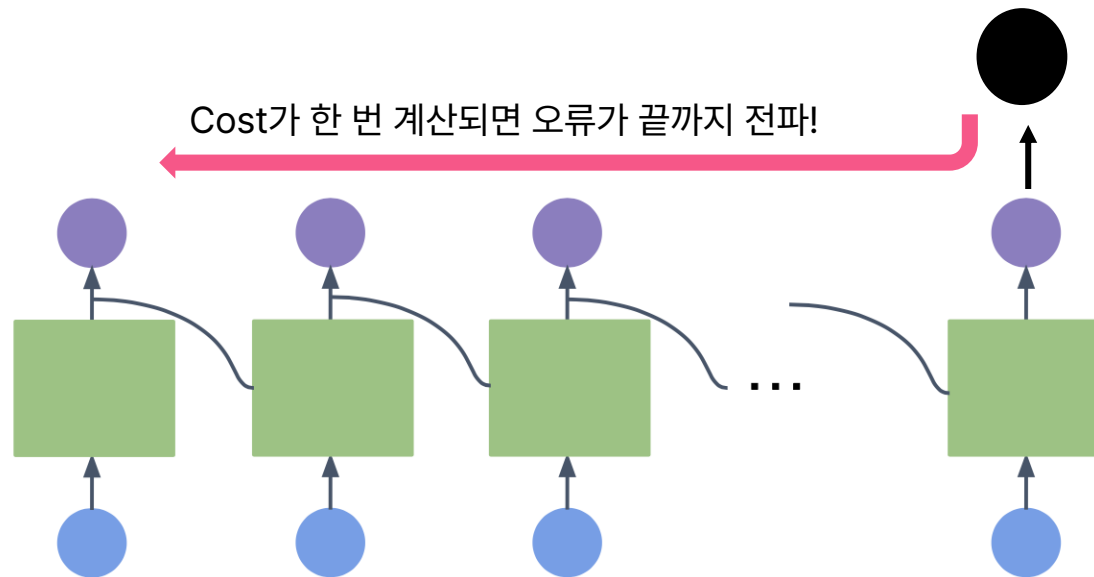
RNN: Computational Graph: Many to Many



- 각 시점별 예측값과 실제값을 통해 시점별 손실 함수값 계산 $\rightarrow L_t$
- L_t 를 모두 더하여 최종 손실값 계산 $\rightarrow L = \text{Sum}(L_t), t = 1 \sim T$

BPTT란?

- 순전파 후, 손실을 최소화하기 위해 역전파를 통해 가중치를 업데이트해야 합니다.
하지만 RNN에서는 모든 이전 은닉 상태가 다음 상태에 영향을 주므로, 기본적인 역전파(Backpropagation)와 달리 시간을 따라 전파되는 형태(BPTT)로 진행됩니다
- BPTT는 RNN의 시간 축 방향 순환 구조를 고려하여, 네트워크의 모든 시점에 대해 가중치를 업데이트하는 알고리즘입니다.



RNN 학습 : Back-propagation Through Time (BPTT)

순전파(Forward Pass)

RNN에서 은닉 상태 h_t 는 다음과 같이 계산됩니다.

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b_h)$$

- W_h : 은닉 상태에서 은닉 상태로 가는 가중치 행렬 (Hidden-to-Hidden)
- W_x : 입력에서 은닉 상태로 가는 가중치 행렬 (Input-to-Hidden)
- b_h : 바이어스
- σ : 비선형 활성화 함수 (tanh 또는 ReLU)

출력 $\hat{y}_t$ 는 은닉 상태의 선형 변환 후 소프트맥스를 적용하여 구해집니다.

$$\hat{y}_t = f(h_t) = W_y h_t + b_y$$

손실 함수는 모든 시점 t 에 대해 다음과 같이 정의됩니다.

$$L = \sum_{t=1}^T \ell(\hat{y}_t, y_t)$$

역전파 (Backward Pass) 개념

- BPTT(Backpropagation Through Time)에서는 손실 함수 L 을 미분하여 가중치 W_h 에 대한 기울기를 계산해야 합니다.
- 우리는 체인 룰을 이용하여 기울기를 전파해야 합니다.
즉, 은닉 상태 h_t 가 과거의 은닉 상태 $h_{t-1}, h_{t-2}, \dots$ 에 의존하기 때문에 기울기 계산이 시간을 거슬러 올라가면서 누적됩니다.
- 가중치 W_h 에 대한 기울기 $\frac{\partial L}{\partial W_h}$ 를 구하는 과정을 살펴보겠습니다

역전파 (Backward Pass) 개념

우선, 특정 시점 t 에서의 손실 ℓ_t 에 대해 가중치 W_h 의 기울기를 구해야 합니다.

$$\frac{\partial \ell_t}{\partial W_h}$$

그러나 은닉 상태 h_t 가 이전 은닉 상태 h_{t-1} 에 의존하기 때문에, 체인 룰을 적용해야 합니다.

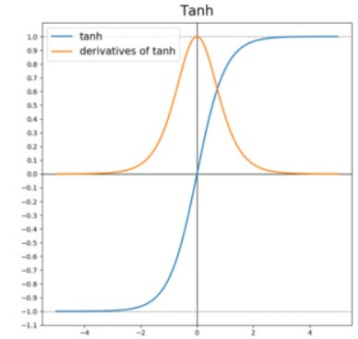
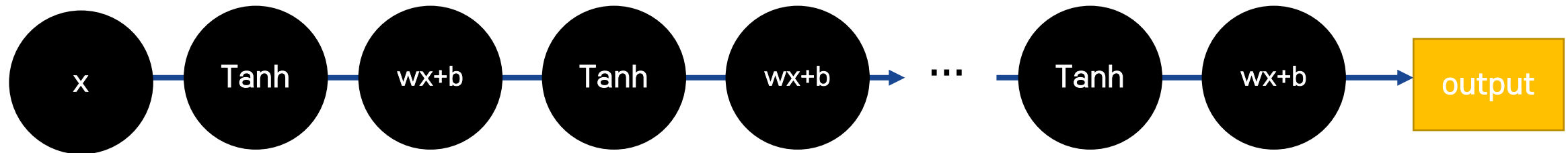
$$\frac{\partial \ell_t}{\partial W_h} = \frac{\partial \ell_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h} + \frac{\partial \ell_t}{\partial h_t} \boxed{\frac{\partial h_t}{\partial h_{t-1}}} \cdot \frac{\partial h_{t-1}}{\partial W_h} + \dots$$

즉, 기울기 전파는 다음과 같이 확산됩니다.

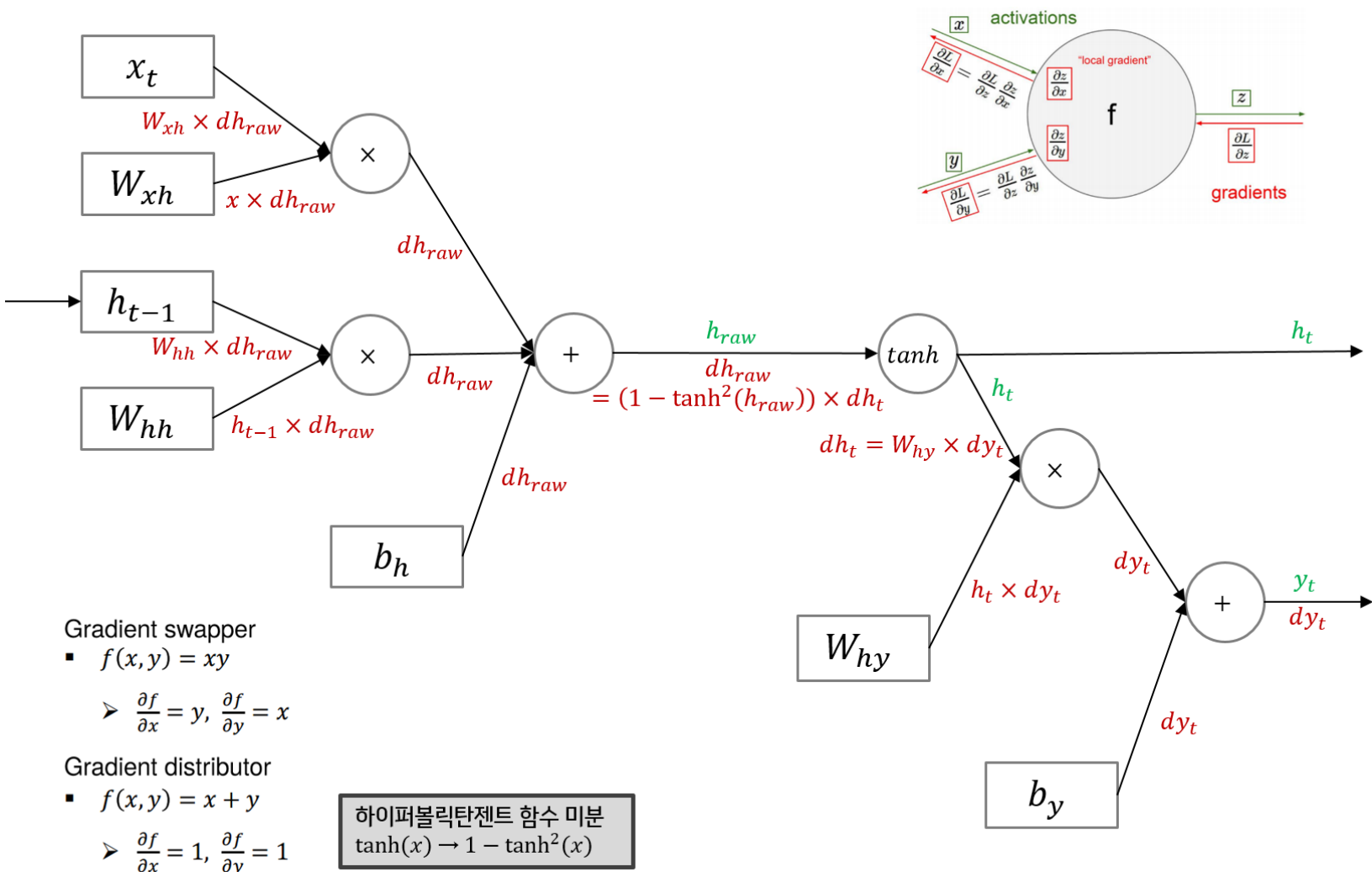
$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^T \left(\frac{\partial \ell_t}{\partial h_t} \cdot \sum_{k=0}^{t-1} \left(\prod_{j=k}^{t-1} \frac{\partial h_{j+1}}{\partial h_j} \right) \frac{\partial h_k}{\partial W_h} \right)$$

RNN 학습 : Back-propagation Through Time (BPTT)

❖ RNN의 역전파

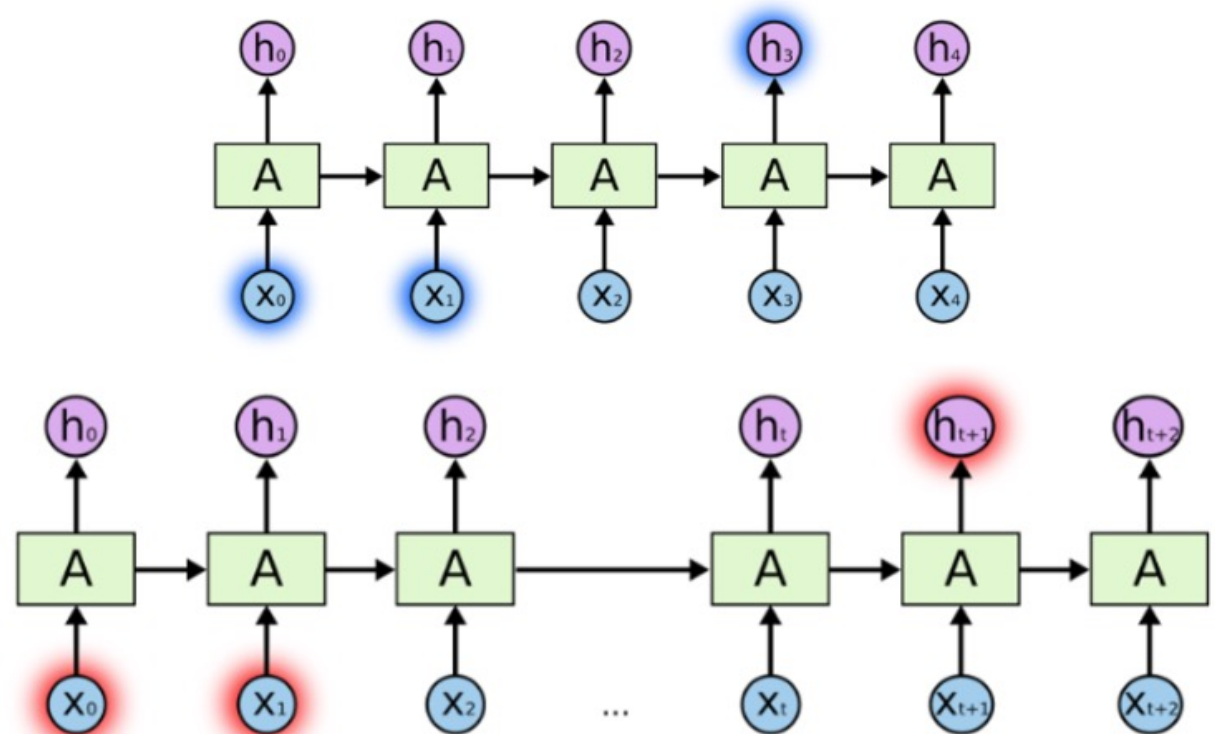


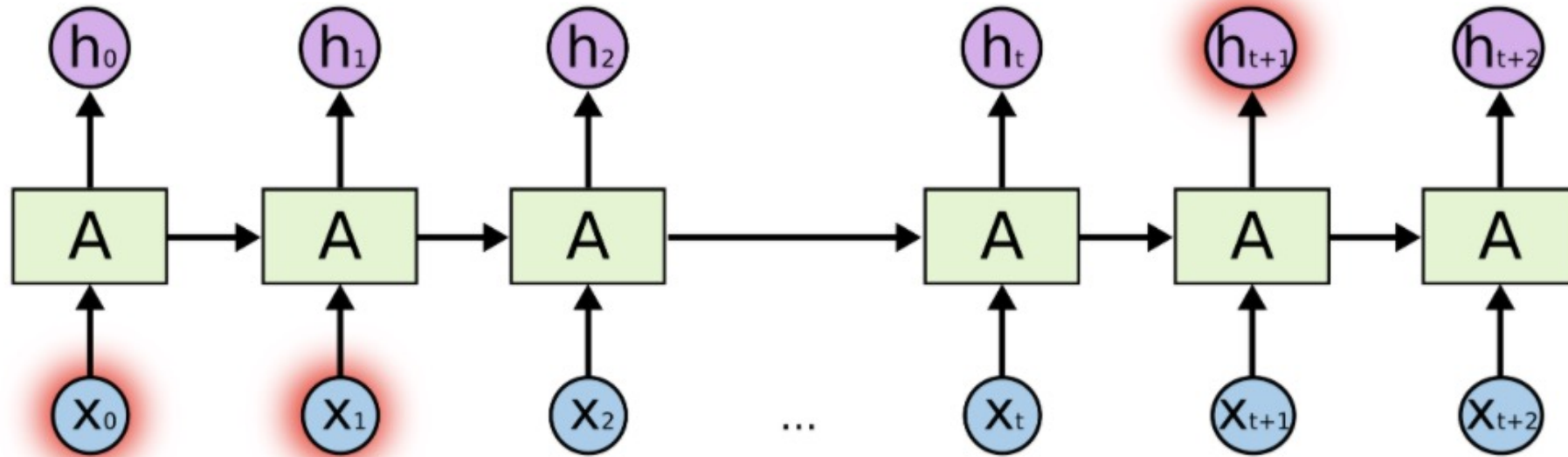
❖ RNN의 역전파



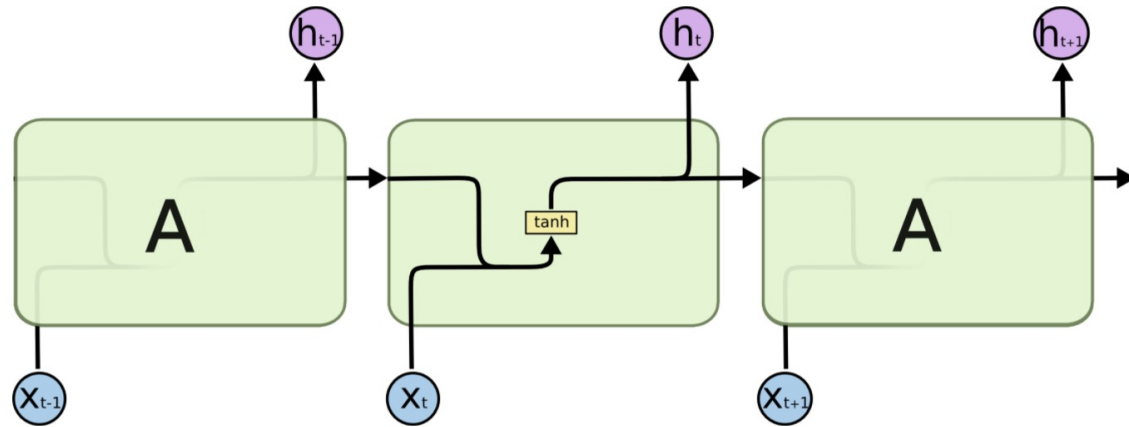
RNN의 문제점

- 기울기 소실 문제: 역전파 과정에서 기울기가 작아짐
- 장기 의존성 문제: 긴 문맥을 학습하는 것이 어려움



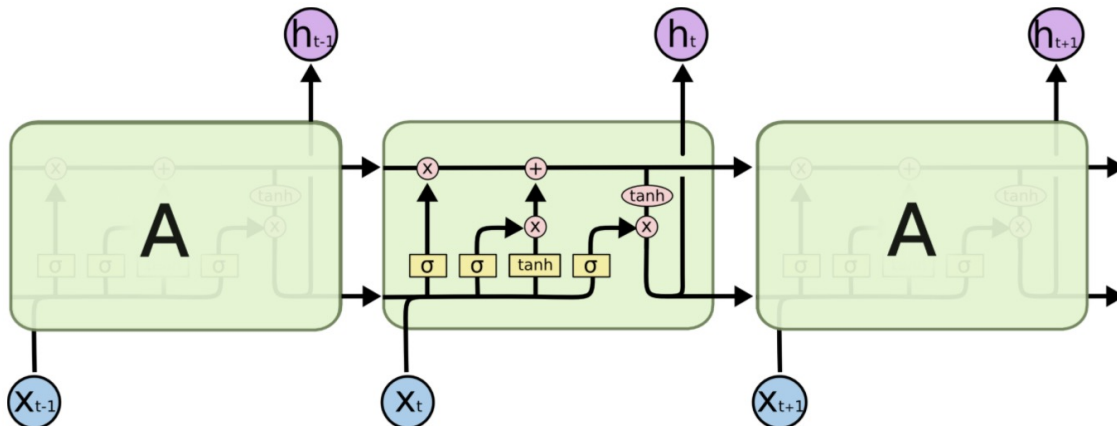


- LSTM (Long Short Term Memory)
 - → 장기 의존성과 단기 의존성을 모두 기억할 수 있다는 의미
- Vanilla RNN의 기울기 소실 문제를 해결하고자 등장

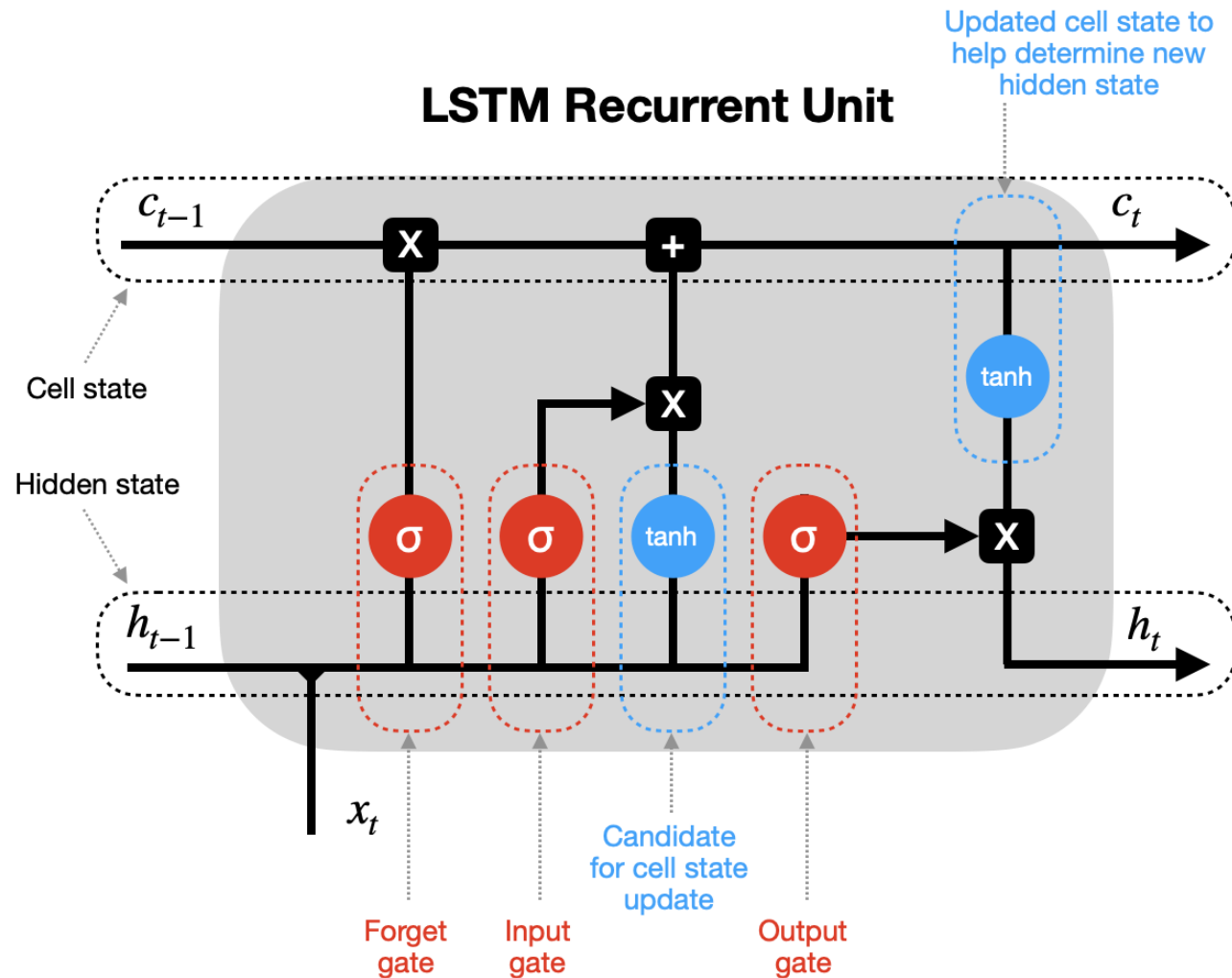


- 기억 소자를 두개로 두자.
- 어떻게?

장기 기억 + 단기 기억

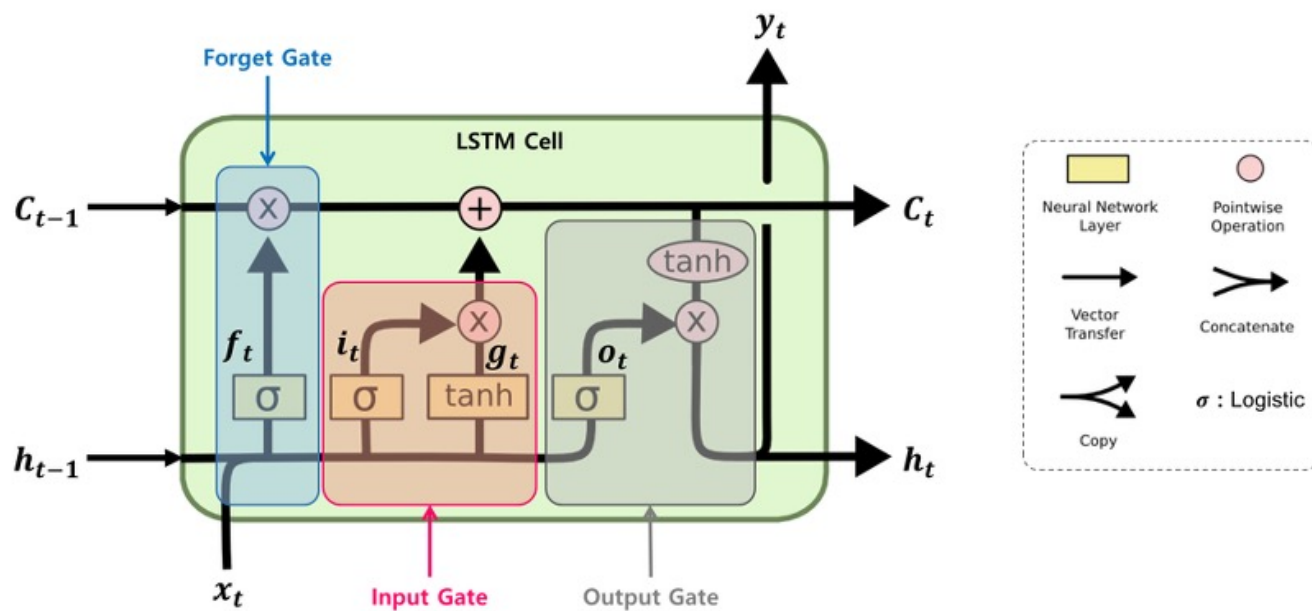


LSTM



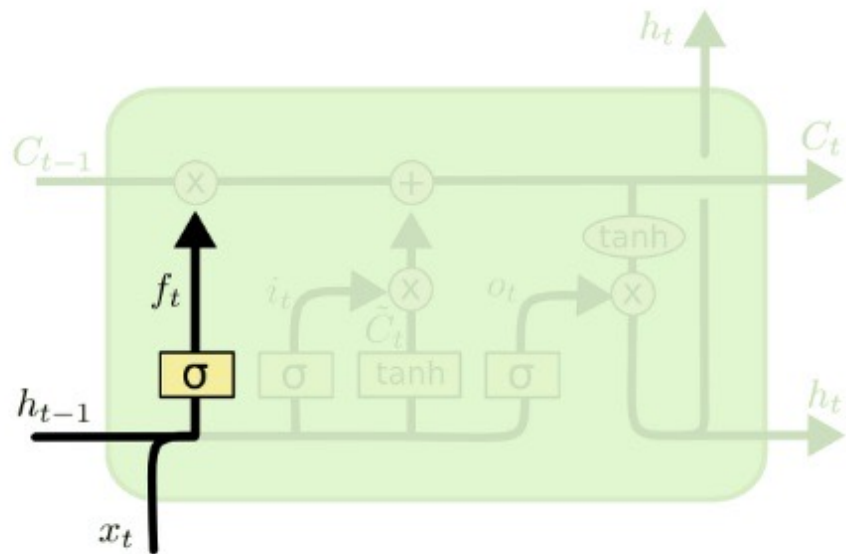
- h_{t-1} - hidden state at previous timestep $t-1$ (short-term memory)
 c_{t-1} - cell state at previous timestep $t-1$ (long-term memory)
 x_t - input vector at current timestep t
 h_t - hidden state at current timestep t
 c_t - cell state at current timestep t
- X** - vector pointwise multiplication **+** - vector pointwise addition
- tanh** - tanh activation function **σ** - sigmoid activation function
- T** - concatenation of vectors
- states** - dashed black outline
gates - dashed red outline
updates - dashed blue outline

Gate



- 3종류의 게이트를 4개의 FC Layer로 구성
- Wf: 망각 게이트 (Forget Gate)
- Wi, WC: 입력 게이트 (Input Gate)
- Wo: 출력 게이트 (Output Gate)

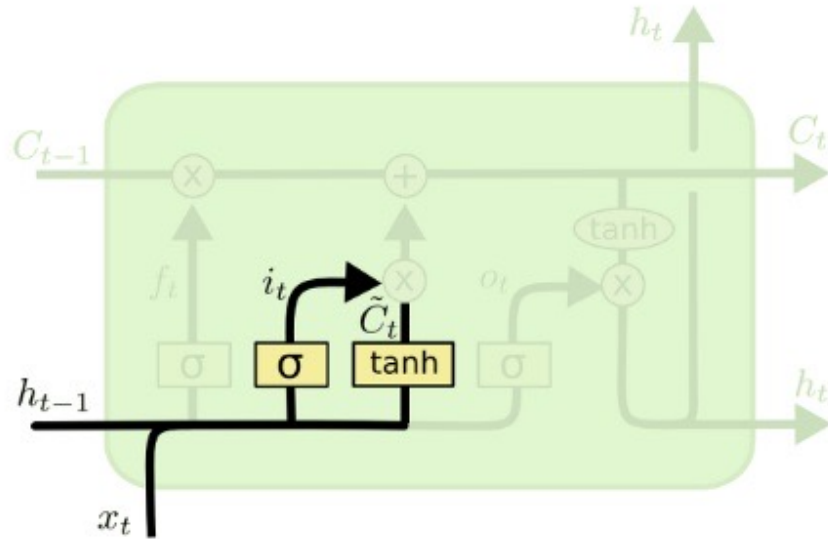
Forget Gate



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- 기존 cell state에서 어떤 정보를 잊을지 결정하는 게이트
- $f_t = \sigma(W_f[h_{t-1}, x_t])$
 - σ : sigmoid 함수 -> 출력범위가 0~1
 - $[h_{t-1}, x_t]$: h_{t-1} 벡터와 x_t 벡터를 concatenate 하는 연산
 - Concatenate 예시: 1 2 3 과 4 5 6 을 concatenate하면 1 2 3 4 5 6
- 입력 : 이전 output, 현재 입력
- 출력 : cell state로 가는 값
 - 1이면 이전 cell state가 그대로 가고, 0이면 이전 cell state는 전부 버려짐

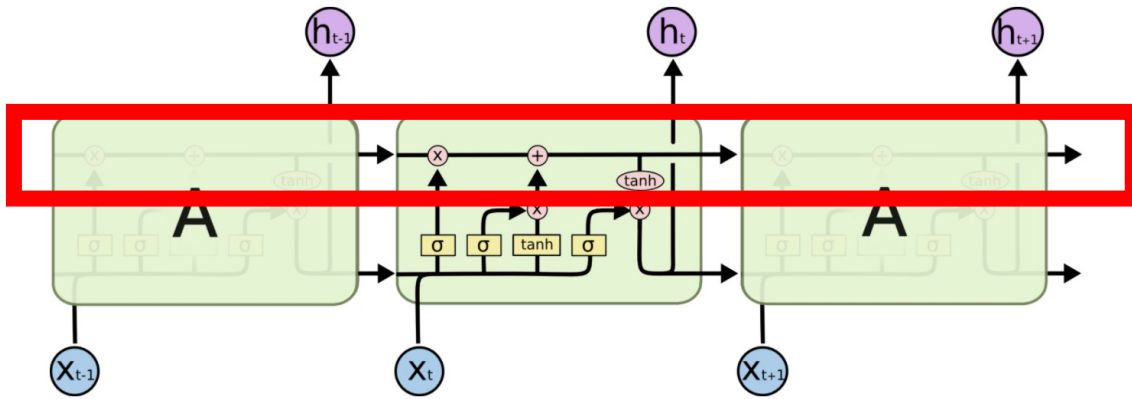
Input Gate



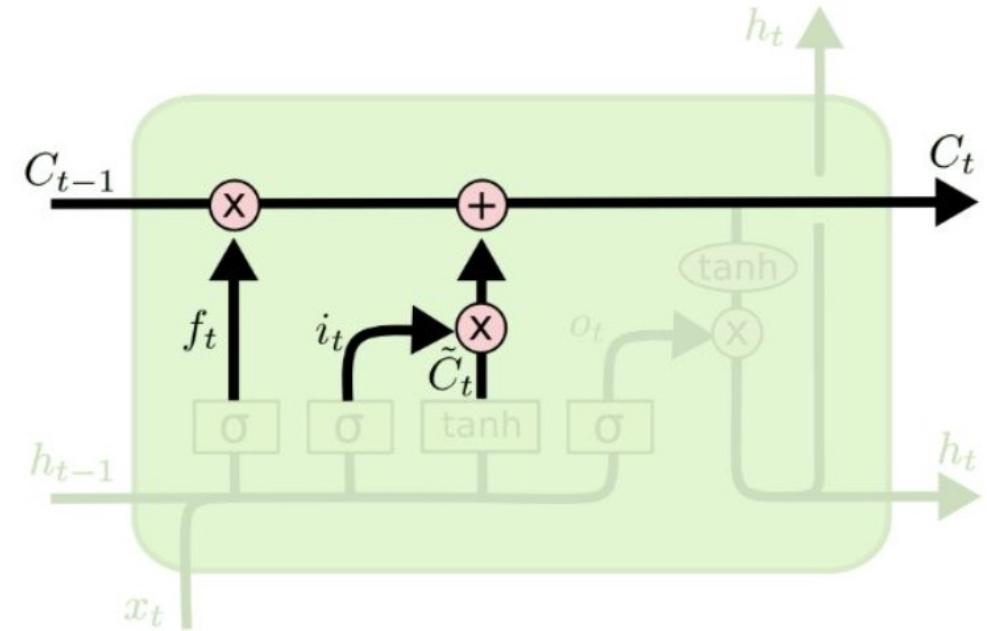
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- 현재 입력 받은 정보에서 어떤 것을 cell state에 저장할지 결정
- sigmoid[0~1]와 tanh[-1~1]를 곱한다.
 - (tanh: 장기 신호의 방향을 결정(저장할 데이터) // sigmoid 입력의 세기를 결정)

Cell State(Update)

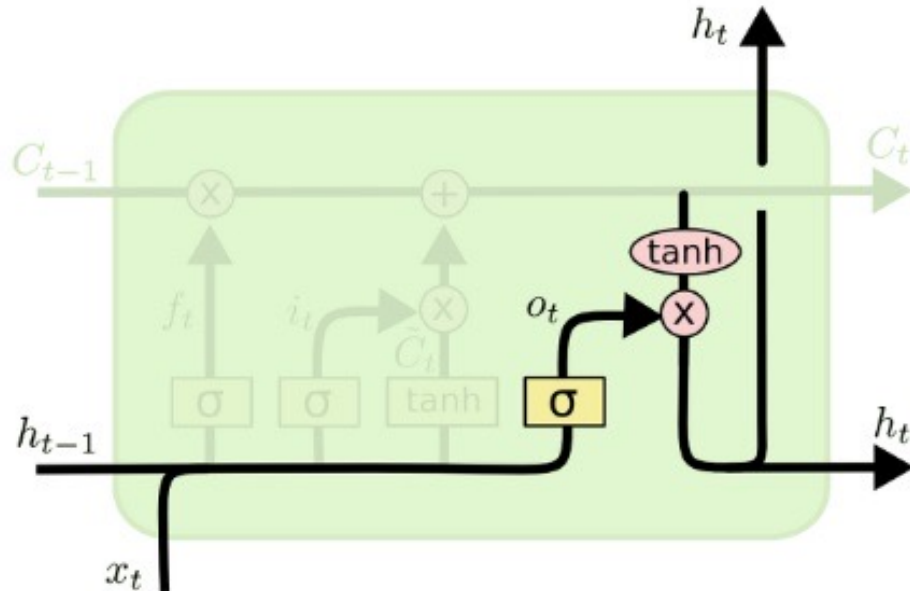


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



- Forget Gate와 Input Gate의 정보를 통해 cell state갱신
- Update : Forget Gate와 Input Gate의 출력 값을 더해주는 부분
- * 연산자는 벡터의 각 원소 별로 곱하는 연산 (Hadamard Product)
 - 예) 1 2 3 * 4 5 6 = 4 10 18

Output Gate

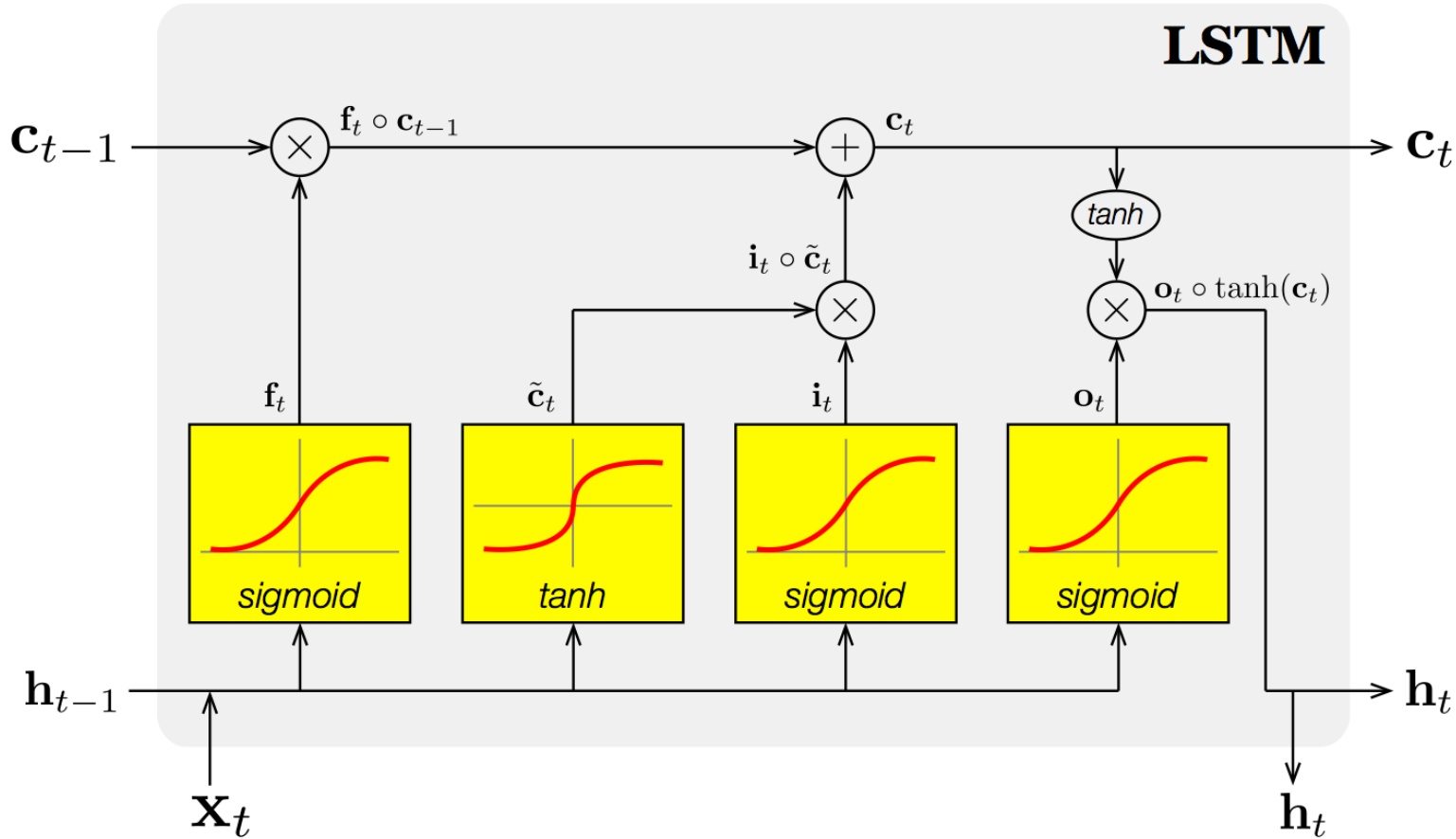


$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

- 다음 hidden state와 출력값을 계산 -> 새로 계산된 cell state를 사용
- 장기기억, 단기 기억, 현재 input을 모두 사용하여 예측 ($h_t == Y_t$)
 - 각각의 비중 $\rightarrow$ weight $\rightarrow$ 학습해야 함

LSTM parameters



Gating variables

$$\begin{aligned}\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \\ \mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \\ \mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)\end{aligned}$$

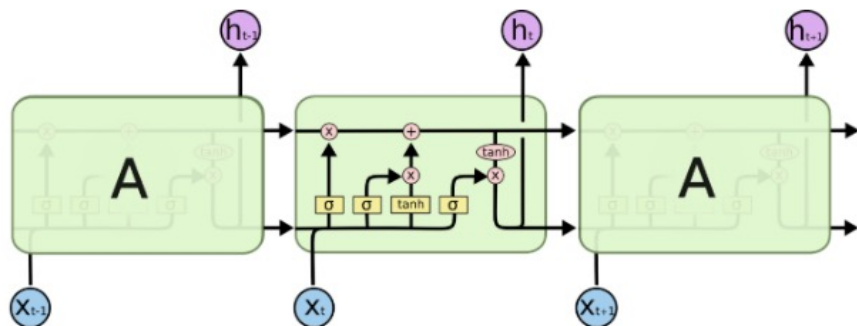
Candidate (memory) cell state

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Cell & Hidden state

$$\begin{aligned}\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t)\end{aligned}$$

Affine 변환 묶기



$$\mathbf{f} = \sigma(\mathbf{x}_t \mathbf{W}_x^{(f)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(f)} + \mathbf{b}^{(f)})$$

$$\mathbf{i} = \sigma(\mathbf{x}_t \mathbf{W}_x^{(i)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(i)} + \mathbf{b}^{(i)})$$

$$\mathbf{g} = \tanh(\mathbf{x}_t \mathbf{W}_x^{(g)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(g)} + \mathbf{b}^{(g)})$$

$$\mathbf{o} = \sigma(\mathbf{x}_t \mathbf{W}_x^{(o)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(o)} + \mathbf{b}^{(o)})$$

$$\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{g} \odot \mathbf{i}$$

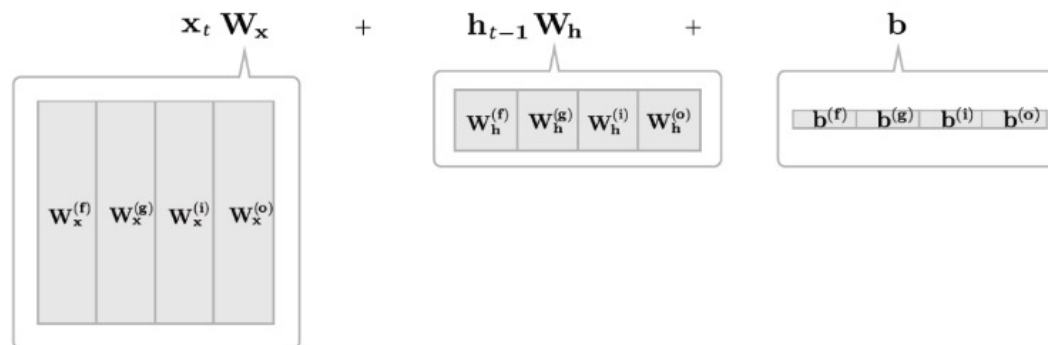
$$\mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{c}_t)$$



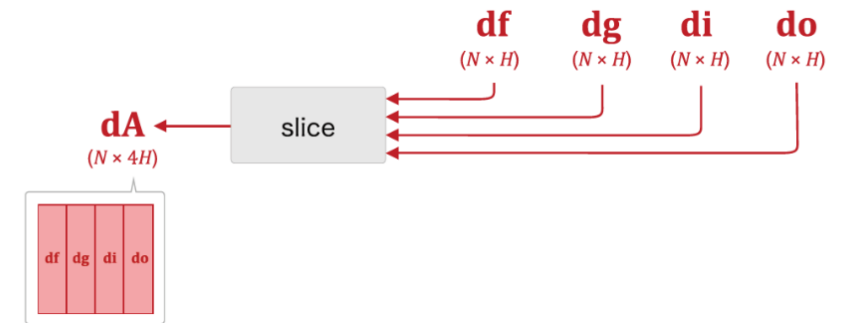
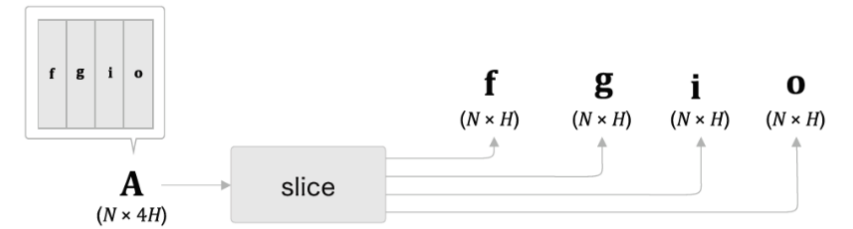
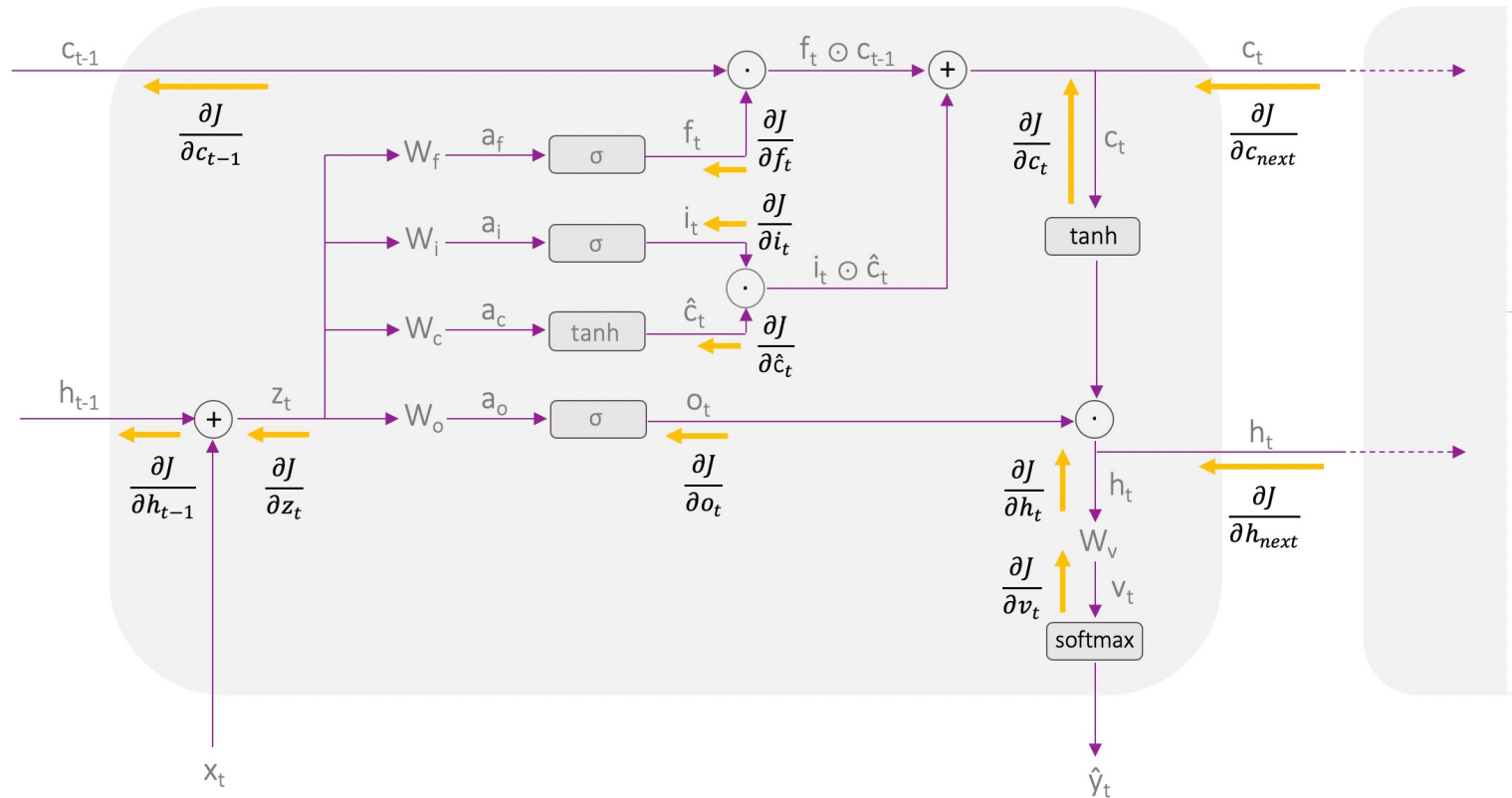
$$\begin{aligned} \mathbf{x}_t \mathbf{W}_x^{(f)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(f)} + \mathbf{b}^{(f)} \\ \mathbf{x}_t \mathbf{W}_x^{(g)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(g)} + \mathbf{b}^{(g)} \\ \mathbf{x}_t \mathbf{W}_x^{(i)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(i)} + \mathbf{b}^{(i)} \\ \mathbf{x}_t \mathbf{W}_x^{(o)} + \mathbf{h}_{t-1} \mathbf{W}_h^{(o)} + \mathbf{b}^{(o)} \end{aligned}$$



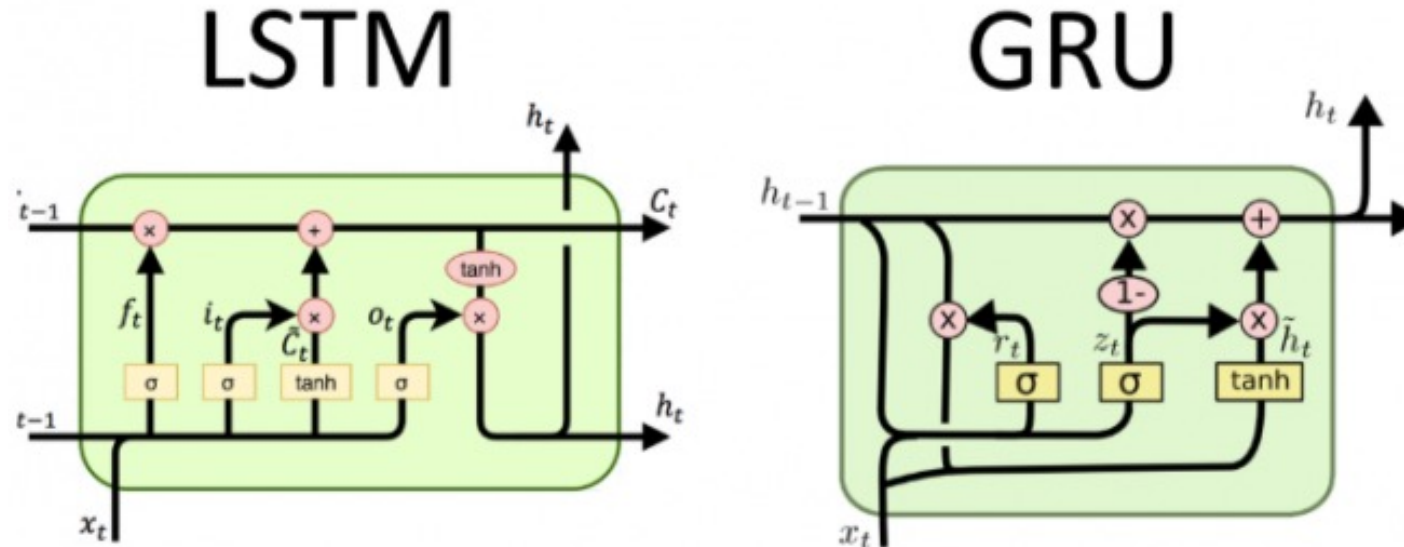
$$\mathbf{x}_t [\mathbf{W}_x^{(f)} \mathbf{W}_x^{(g)} \mathbf{W}_x^{(i)} \mathbf{W}_x^{(o)}] + \mathbf{h}_{t-1} [\mathbf{W}_h^{(f)} \mathbf{W}_h^{(g)} \mathbf{W}_h^{(i)} \mathbf{W}_h^{(o)}] + [\mathbf{b}^{(f)} \mathbf{b}^{(g)} \mathbf{b}^{(i)} \mathbf{b}^{(o)}]$$



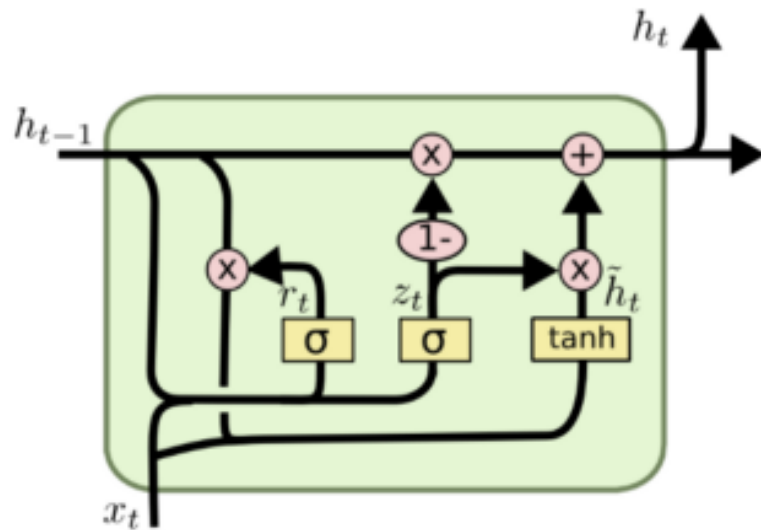
LSTM의 학습



$$c^{(t)} = f^{(t)} \odot c^{(t-1)} + i^{(t)} \odot \tilde{c}^{(t)} \quad h^{(t)} = o^{(t)} \odot \tanh c^{(t)}$$



- Gated Recurrent Unit의 약자
- LSTM이 가지는 3개의 게이트를 2개로 간소화하고 Cell State를 없앴
 - 파라미터 수가 감소하여 LSTM보다 빠른 학습 속도를 가짐
 - 그럼에도 음악 모델링, 음성 신호 모델링 및 자연어 처리의 특정 작업에 대한 GRU의 성능은 일반적으로 LSTM과 비슷한 수준
- 마찬가지로 새로 계산된 hidden state를 출력값으로도 사용



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

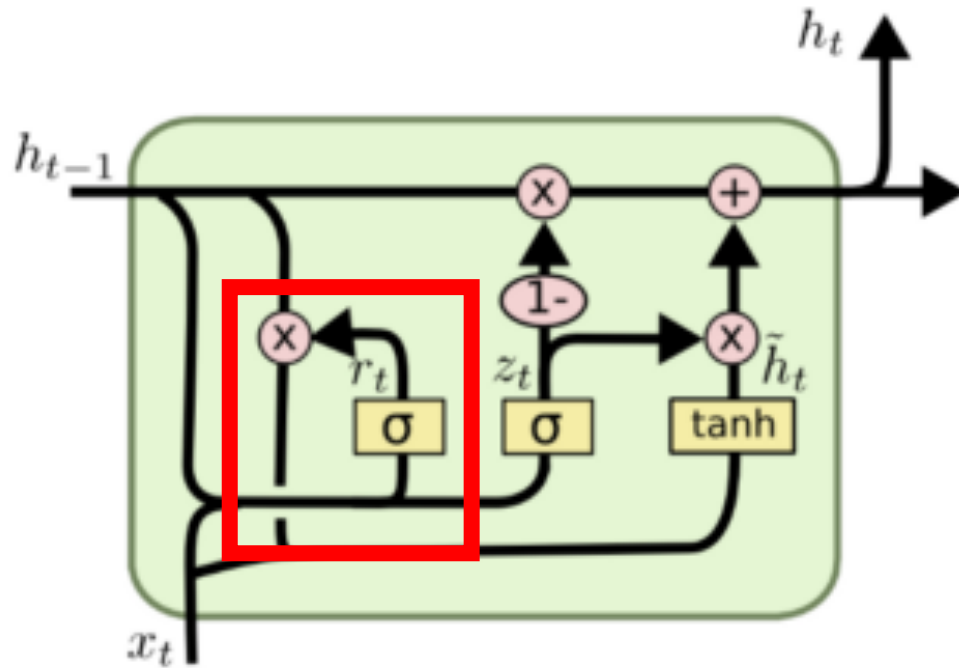
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- 2종류의 게이트를 2개의 FC Layer로 구성
 - W_r : 리셋 게이트 (Reset Gate)
 - W_z : 업데이트 게이트 (Update Gate)

Reset Gate



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

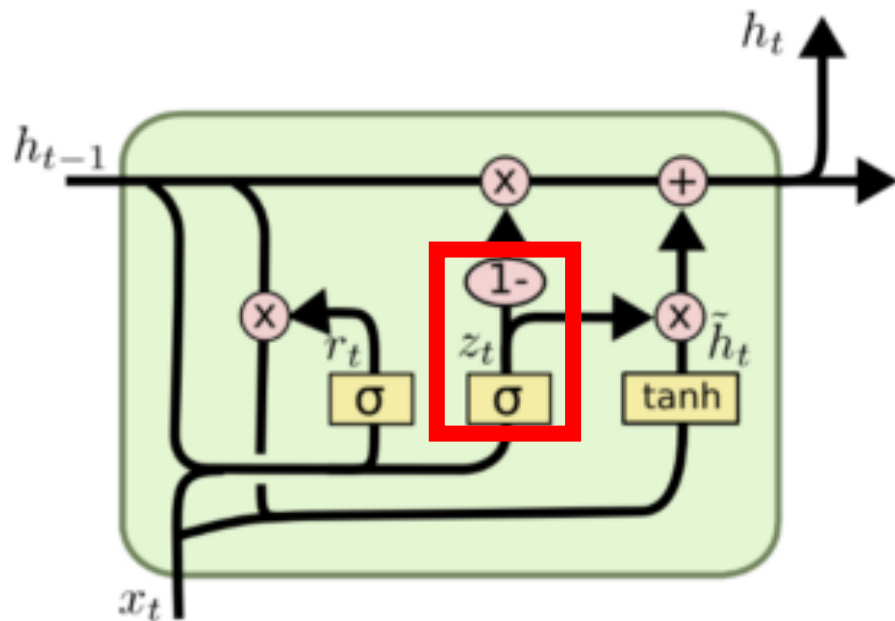
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- 기존 hidden state의 정보를 얼마나 초기화할 지 결정하는 게이트
- $r_t = \sigma(W_r[h_{t-1}, x_t])$
 - Sigmoid -> 0~1 사이 값으로 출력

Update Gate



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

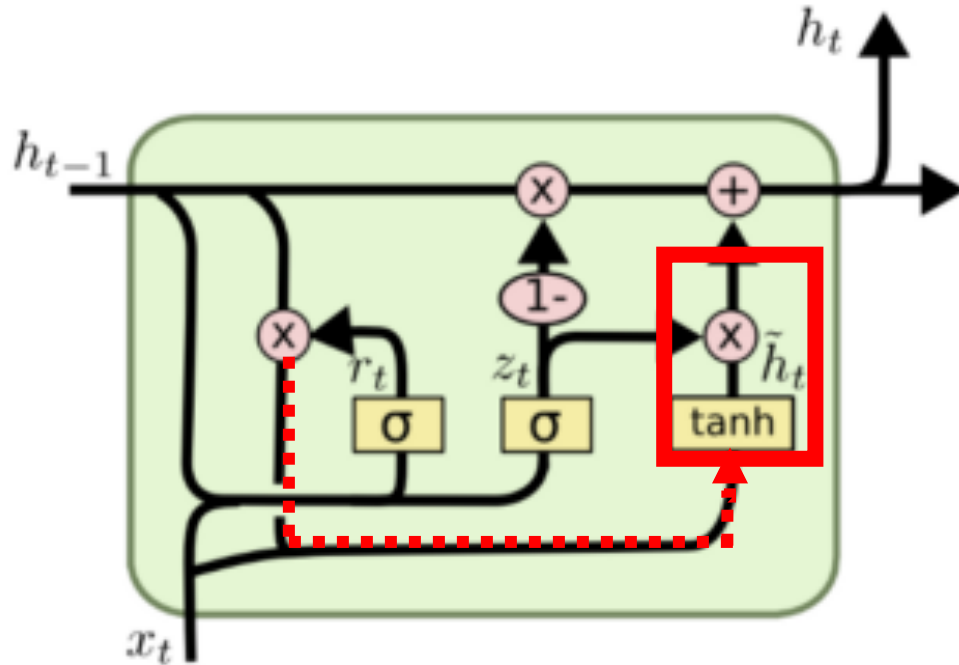
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- 기존 hidden state의 정보를 얼마나 사용할 지 결정하는 게이트
 - 과거와 현재의 정보 업데이트 비율을 결정하는 게이트
- $z_t = \sigma(W_z[h_{t-1}, x_t])$
 - Sigmoid -> 0~1 사이 값으로 출력

새로운 Hidden State 계산



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

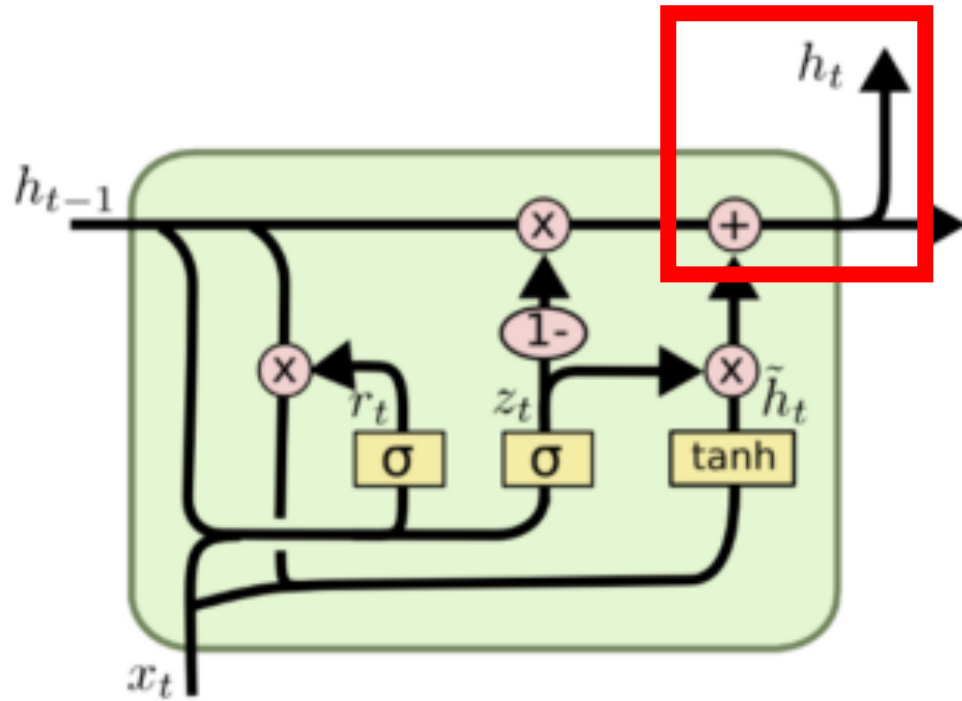
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- Reset Gate의 결과를 통해 새로운 hidden state의 후보를 계산
- $\tilde{h}_t = \tanh(W_i[r_t * h_{t-1}, x_t])$
 - $r_t * h_{t-1}$ 부분이 점선을 통해 W_i 로 전달됨

새로운 Hidden State 계산



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

- Update Gate의 결과를 통해 새로운 hidden state를 계산
- Update Gate의 정보만이 새로운 hidden state 계산에 사용됨
- Update Gate가 LSTM의 Forget Gate와 Input Gate를 하나로 합친 것과 유사한 역할

- GRU와 LSTM 중 어떤 것이 모델의 성능 면에서 더 낫다고 단정지어 말할 수 없으며, 기존에 LSTM을 사용하면서 최적의 하이퍼 파라미터를 찾아낸 상황이라면 굳이 GRU로 바뀌서 사용할 필요는 없음
- 경험적으로 데이터 양이 적을 때는 매개 변수의 양이 적은 GRU가 조금 더 낫고, 데이터 양이 더 많으면 LSTM이 더 낫다고도 함
- GRU보다 LSTM에 대한 연구나 사용량이 더 많은데, 이는 LSTM이 더 먼저 나온 구조이기 때문

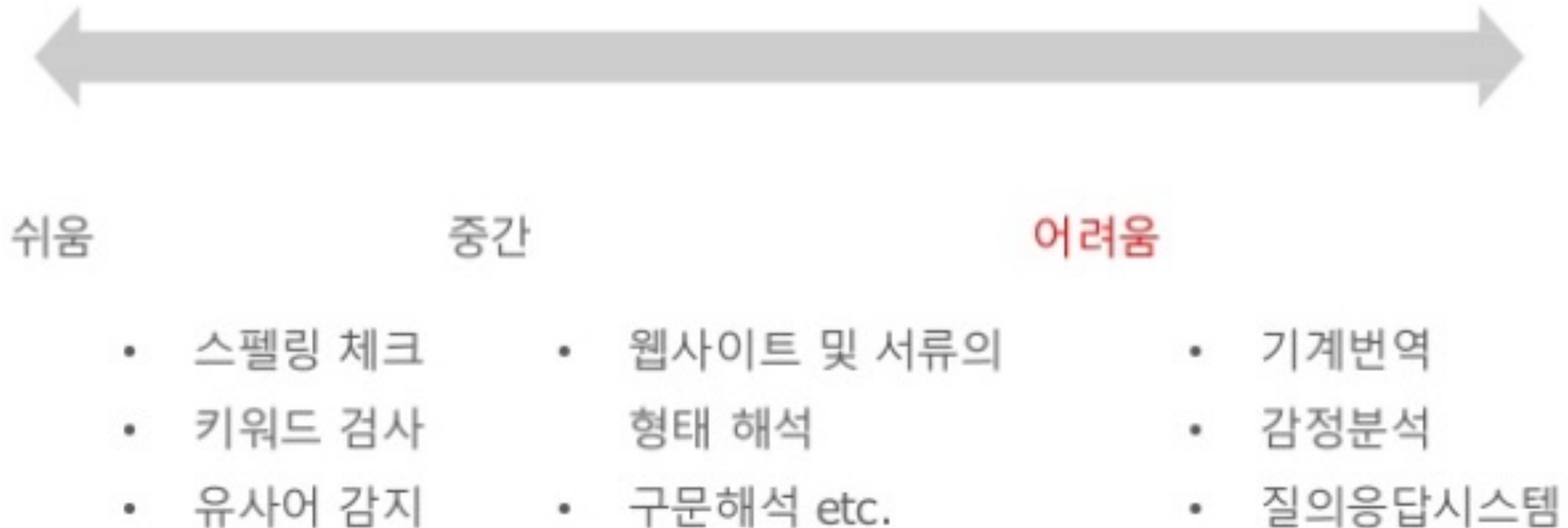
자연어란?

인간인 우리가 대화, 서류, 채팅 등을 통해 일상적으로 사용하는 언어를 '자연어 (Natural language)'라고 합니다.

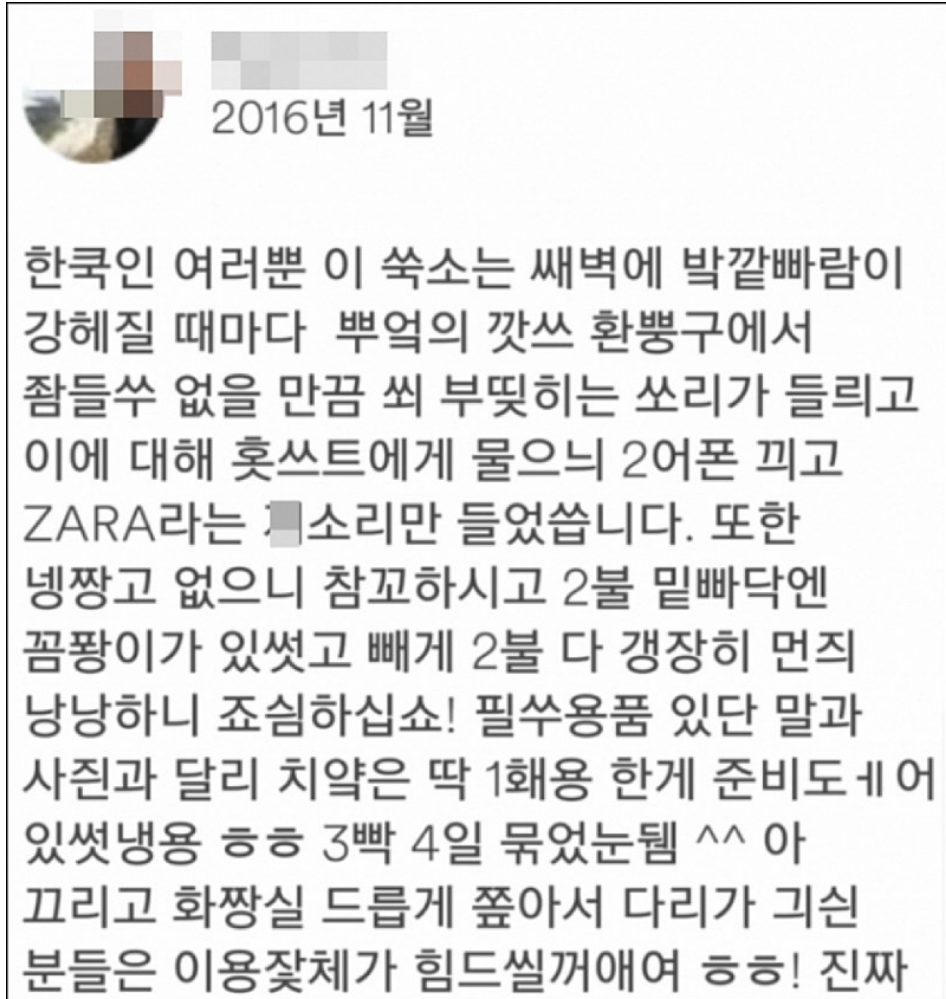


자연어 처리(Natural Language Processing)란?

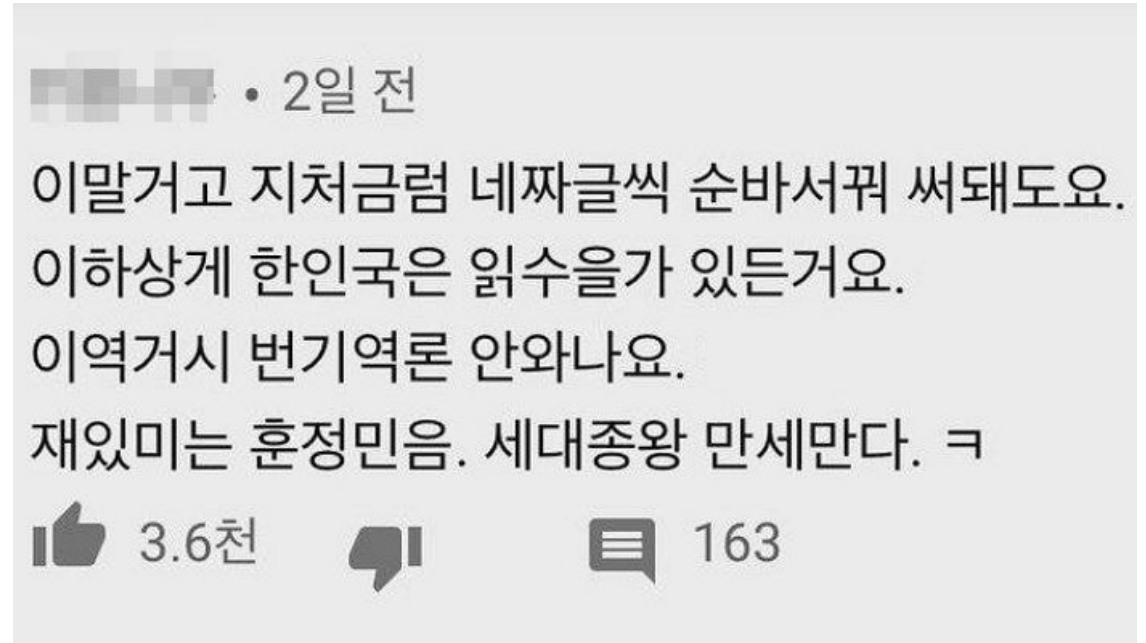
자연어 처리(Natural Language Processing)는
자연어의 의미를 분석하여 컴퓨터가 처리할 수 있도록 하는 일을 말한다



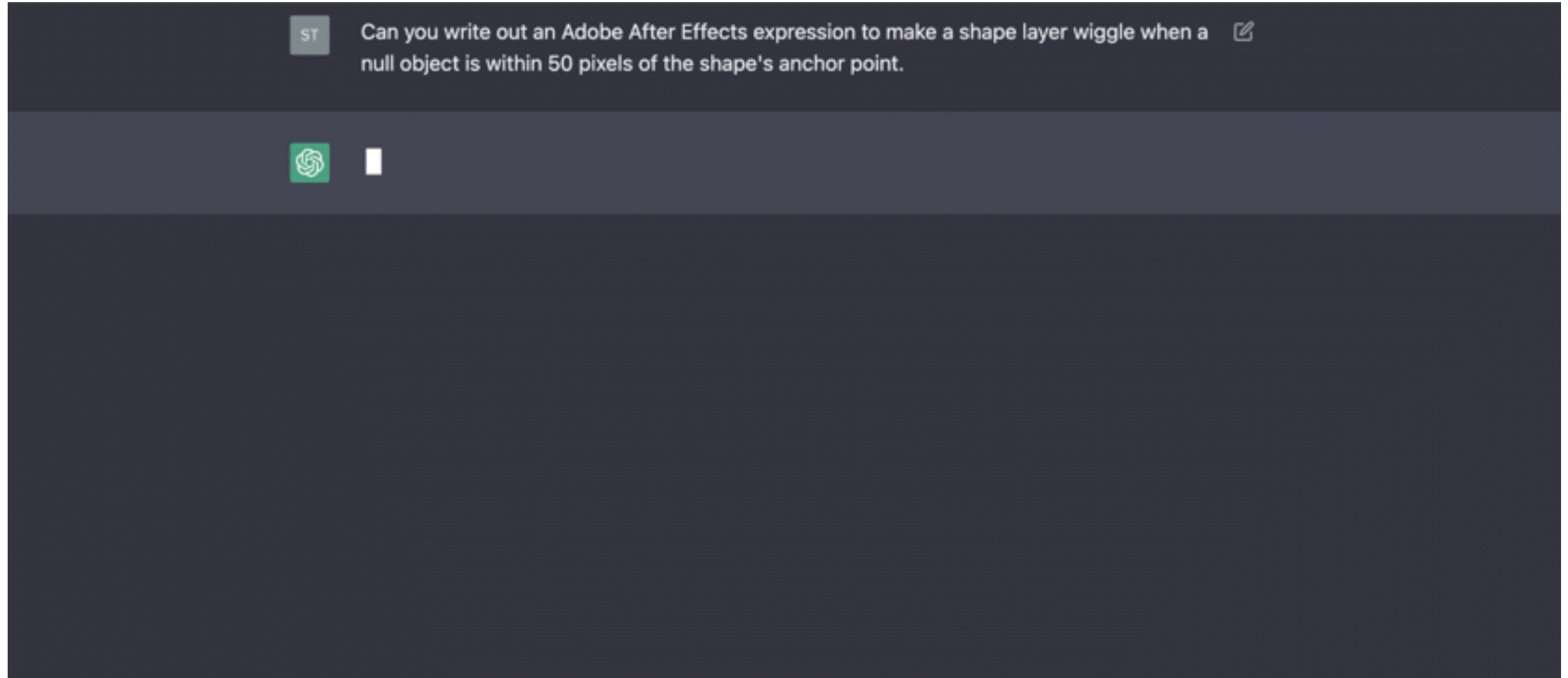
자연어 처리(Natural Language Processing)란?

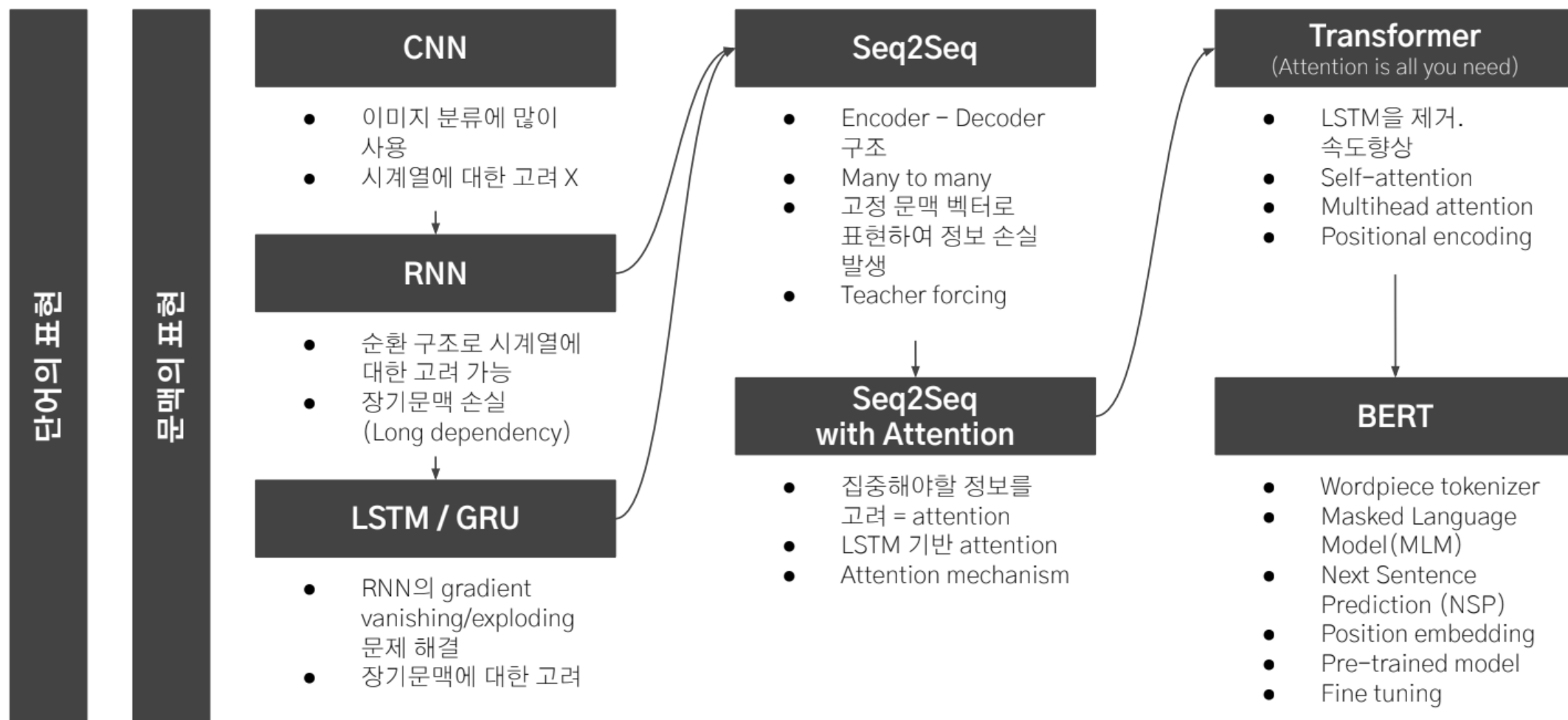


<https://ellatv.tistory.com/273>



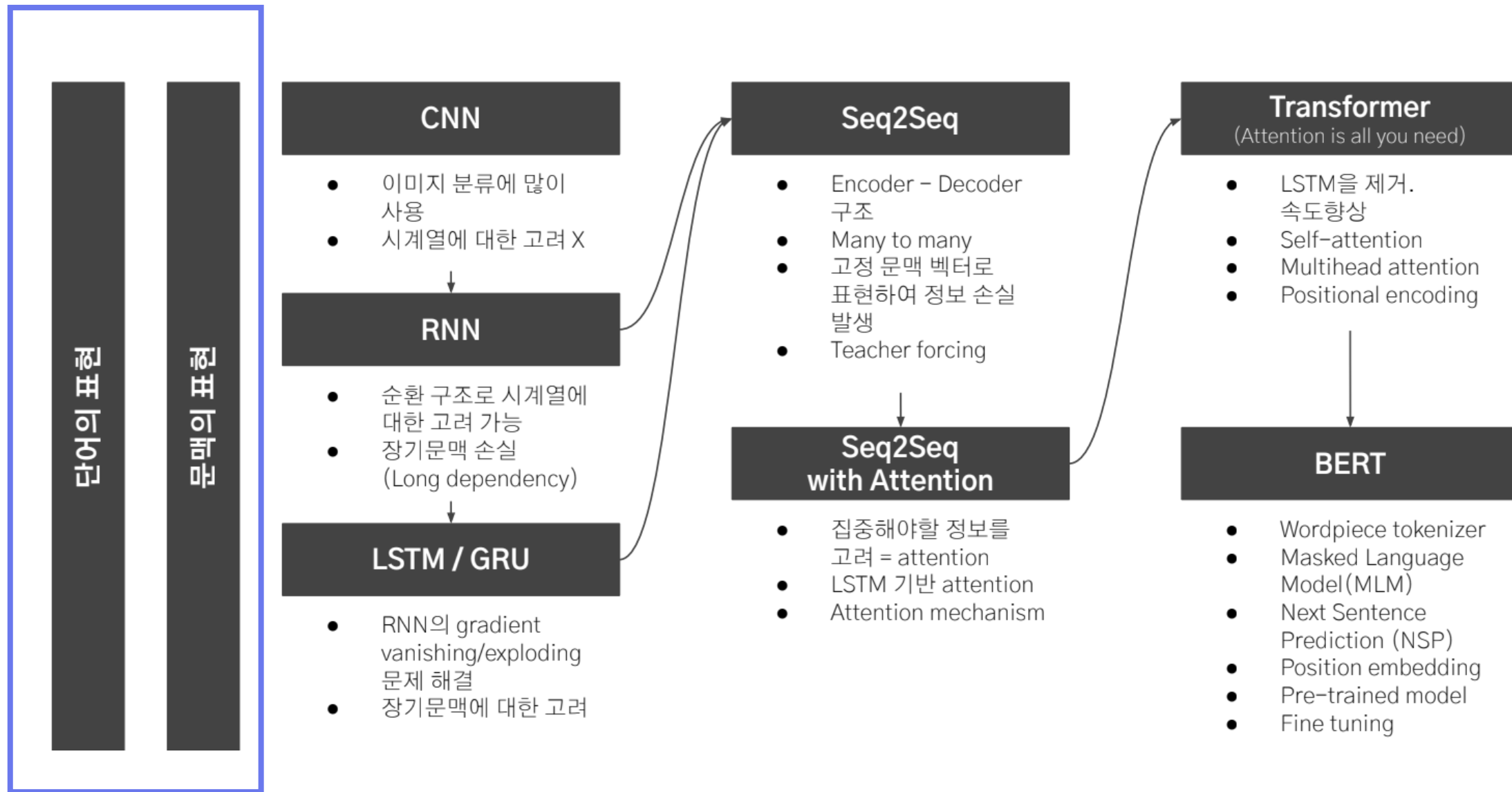
<https://www.insight.co.kr/news/225380>



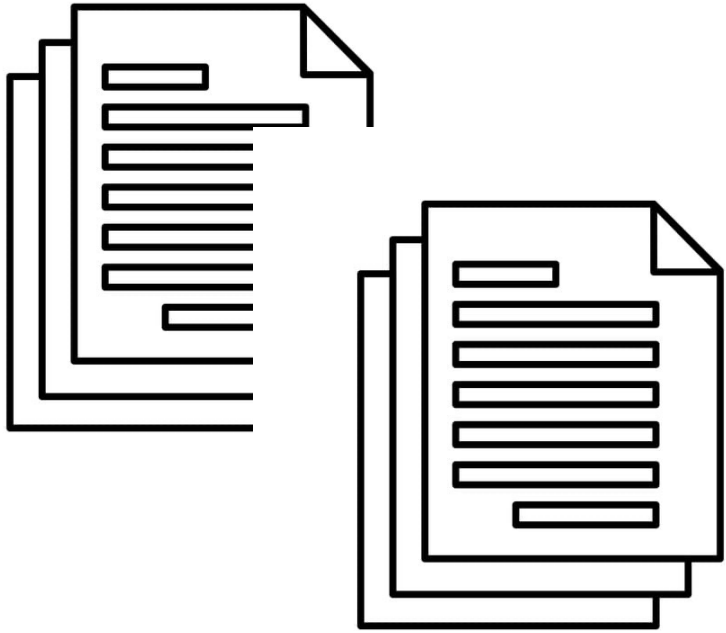


자연어 처리를 시작해봅시다.
어디서부터 시작해야 할까요?

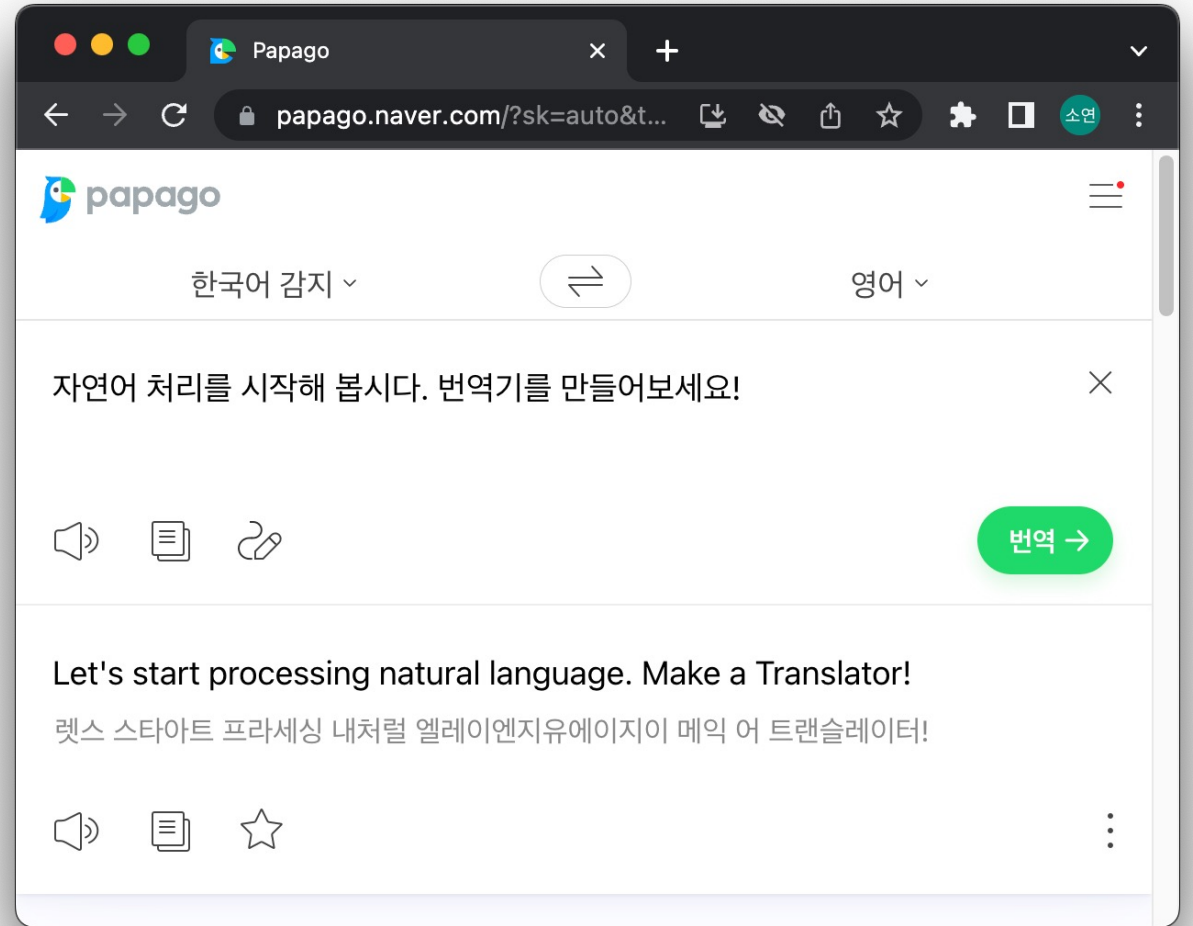
자연어 처리(NLP)의 시작



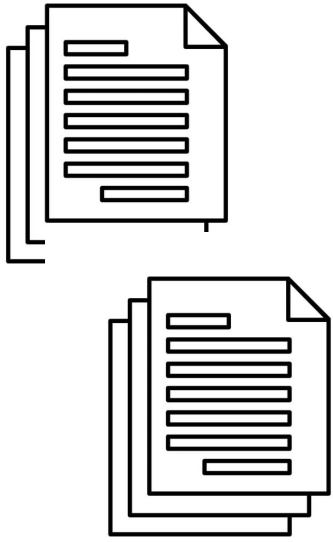
자연어 처리(NLP)의 시작



한국어 문장 10만개
영어 문장 10만개



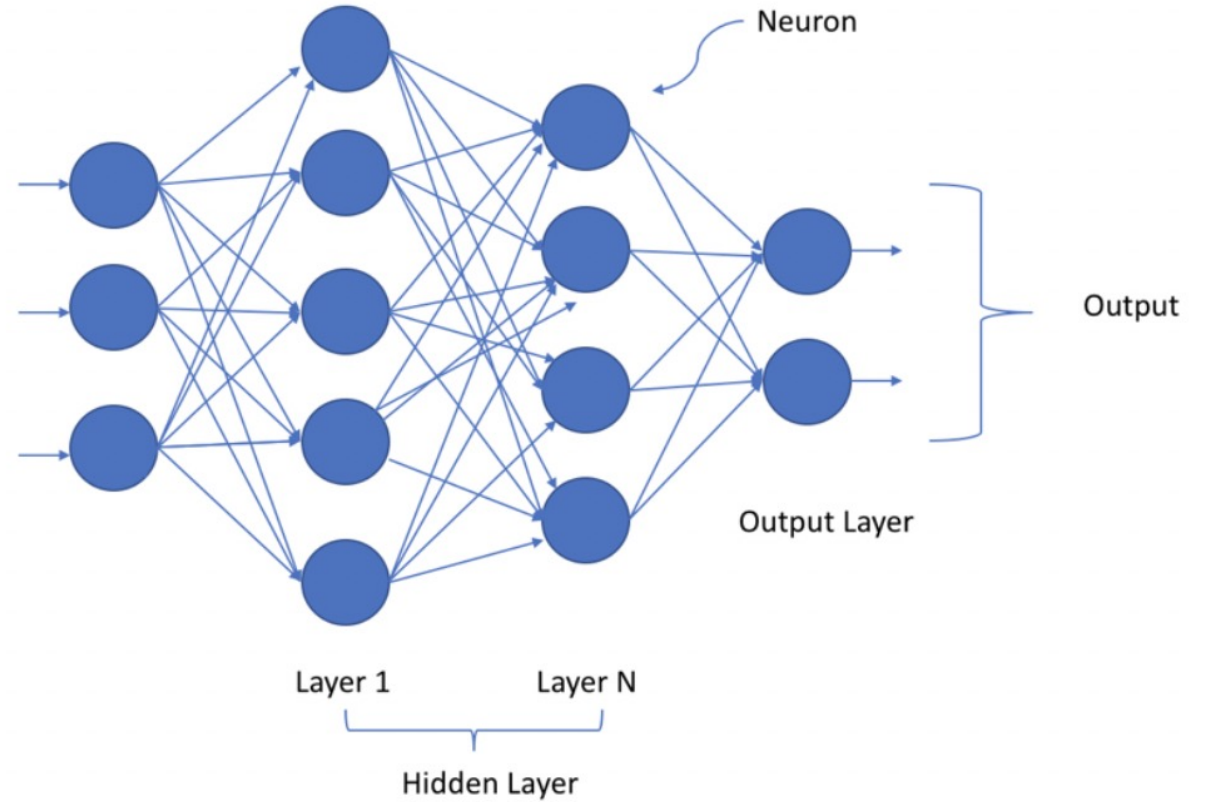
자연어 처리(NLP)의 시작



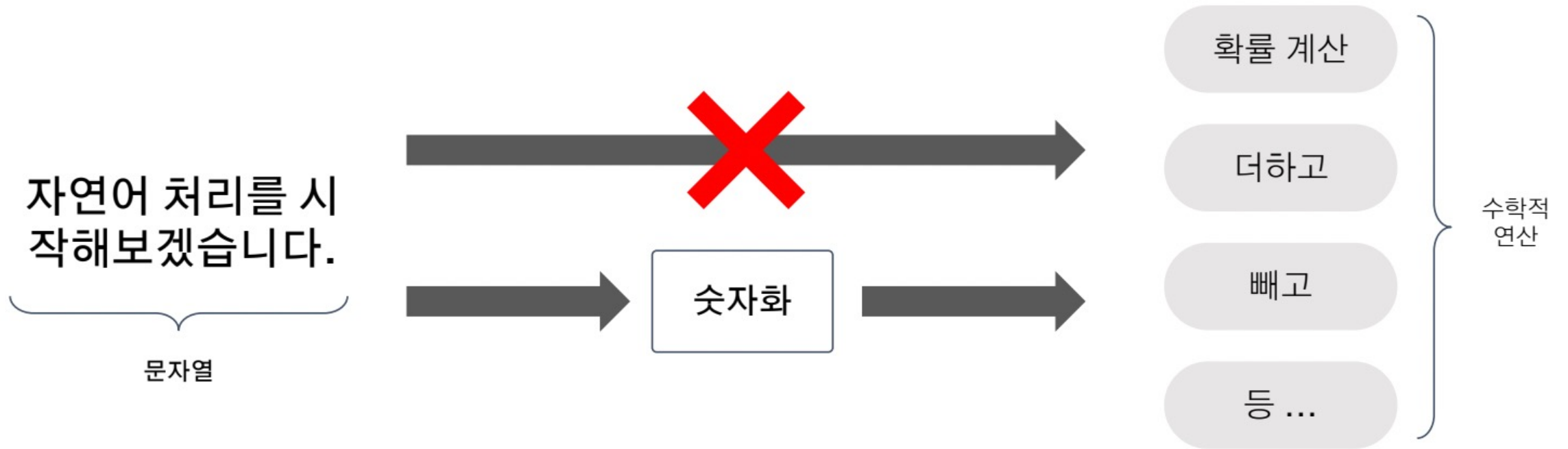
한국어 문장 10만개
영어 문장 10만개



Input Data



자연어 처리(NLP)의 시작



- NLP는 자연어를 다루는 분야로 문서요약, 감성분석, 질의응답, 문장 생성을 아우른다
- 자연어처리를 하기 전에 가장 먼저, 컴퓨터는 어떻게 언어를 이해할까?
 - 컴퓨터는 언어를 "숫자"로 이해한다
 - 언어를 숫자로 표현하는 것을 임베딩(Embedding)이라 한다.
 - 단어에 무작위 숫자를 붙이는 방법 → 유사한 단어에 유사한 숫자를 부여하는 방법 → 유사한 문장에 유사한 숫자를 부여하는 방향으로 발전하고 있다
- 언어처리에서 가장 중요한 개념 중 하나가 임베딩이다!
- 서로 다른 모델은 서로 다른 임베딩 방법을 갖고 있다.

Red
Green
Blue

0
1
2

Label Encoding

1	0	0
0	1	0
0	0	1

One-Hot Encoding

One-Hot Encoding의 문제점

1. 차원 크기의 문제

Google

한국 단어 개수

전체 이미지 뉴스 동영상 지도 더보기 도구

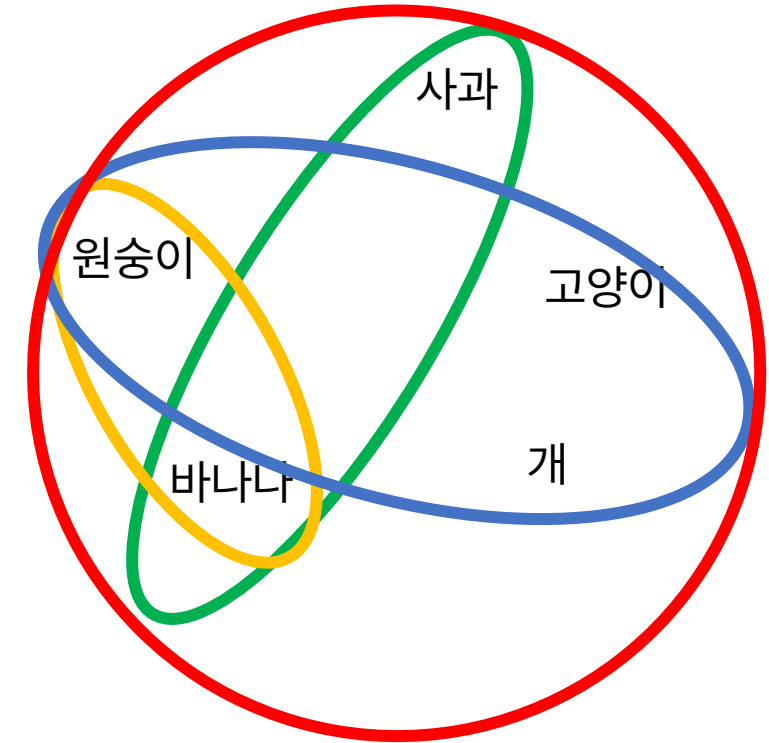
검색결과 약 938,000개 (0.41초)

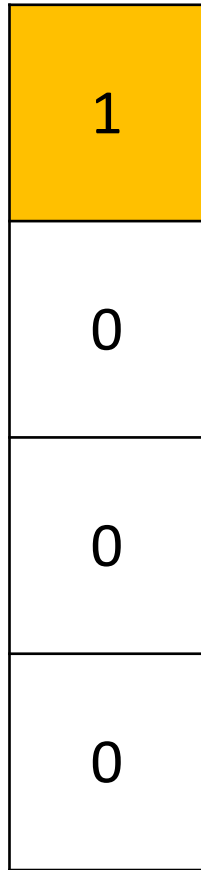
현재 표준국어대사전 표제어 개수는 **511,348개**입니다. 옛말의 개수는 12,481개입니다. 전체 개수에서 옛말의 개수를 빼면, 498,867개입니다. 보기 2560. 7. 4.

<https://www.korean.go.kr> > onlineQna > onlineQnaView

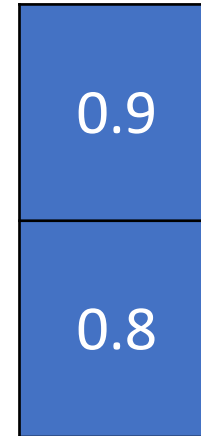
온라인가나다 상세보기(한글 단어 개수에 대한 질문입니다.)

2. 의미를 담지 못하는 문제





Red



Red

나는 아침에 회사에 간다

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

희소행렬(Sparse Matrix)



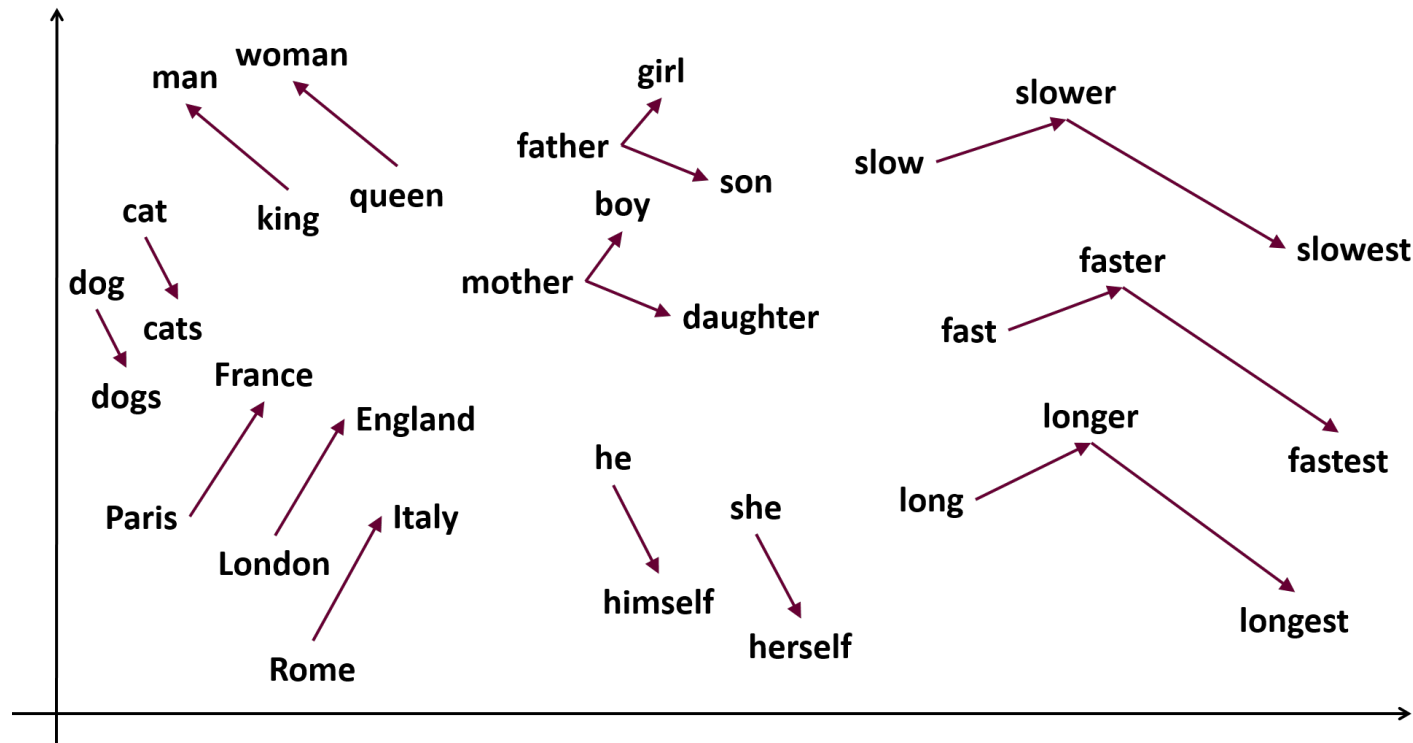
나는 아침에 회사에 간다

0.9	0.3	0.2	0.1
0.8	0.1	0.08	0.2

밀집행렬(Dense Matrix)

Embedding Vector

- 원-핫 인코딩(One-Hot Encoding)으로 표현된 희소 벡터는 차원이 매우 크고 대부분이 0으로 채워져 비효율적입니다.
- 임베딩 레이어는 단어의 의미적 관계를 학습하여 벡터 공간에서 의미적으로 유사한 단어가 가까워지도록 합니다.
- 예: "apple"과 "banana"는 서로 가까운 벡터로, "apple"과 "car"는 멀리 떨어진 벡터로 표현됩니다.



임베딩 벡터 학습 방법

- 사전 학습된 임베딩(Pre-trained Embeddings) 사용:
 - word2vec, GloVe 등 미리 학습된 임베딩을 가져와 모델에 적용
 - 장점: 학습 속도 단축, 더 나은 성능 가능
- 훈련 중 학습(Learned During Training):
 - 모델이 훈련 데이터에 맞는 임베딩 벡터를 직접 학습.
 - 장점: 문제에 특화된 임베딩 생성 가능.

Word → Integer → lookup Table → Embedding vector



임베딩 레이어(Embedding Layer)

임베딩 레이어는 신경망의 입력 계층 중 하나로, 단어를 고차원 공간의 실수 벡터로 매핑하는 역할을 합니다.

- 입력: 단어의 고유 인덱스(정수값)
- 출력: 해당 단어의 임베딩 벡터 (예: [0.1, 0.5, 0.3, 0.7])

```
[ [0.2, 0.4, ..., 0.1], # "I"  
  [0.6, 0.8, ..., 0.5], # "love"  
  [0.3, 0.1, ..., 0.7]] # "NLP"
```

감사합니다 😊